# Chapter 3
# Software Engineering

**Abstract** In this chapter we discuss software construction tools including the venerable GNU Compiler Collection (GCC) compilers, source code configuration systems (CVS, SVN and git), as well as the GNU Build Tools. Automatic build management tools GNU `gmake` and SCons are described with examples. The Bugzilla defect tracking system is described; incidentally, this book had a Bugzilla page during its development for issue tracking. Source code editors and IDEs (including Emacs, Kdevelop and Eclipse) are shown as well as debugging, documentation and profiling tools.

## Contents

**3.1 GCC : GNU Compiler Collection ..... 47**
**3.2 Source Code Configuration Systems ..... 59**
**3.3 GNU Build System ..... 66**
**3.4 Automatic Build Dependency Management ..... 69**
**3.5 Bugzilla : Defect Tracking System ..... 75**
**3.6 Editing Source Code ..... 77**
**3.7 Static Checks on Source Code ..... 81**
**3.8 GNU `gcov`: Test Coverage Program ..... 82**
**3.9 Debug Tools ..... 87**
**3.10 Doxygen ..... 90**
**3.11 Source Navigation ..... 92**
**3.12 Profilers ..... 96**
**3.13 Conclusions ..... 103**

## 3.1 GCC : GNU Compiler Collection

GCC stands for the GNU Compiler Collection (and not the GNU C Compiler), as today it supports many languages in addition to C. In this Section we describe GCC (as of version 4.5.0), and its command-line options, extensions, and optimizations. The key features of GCC in the recent years have been support of modern architectures with super-scalar instruction issue, parallelism (auto-SIMD as well as support

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_3, © Springer Science+Business Media, LLC 2011

for OpenMP) and more complete support of C++ features, link-time optimization, and with version 4.5.0, support of *plugins*. GCC is the official compiler of the GNU project and is thus used to build the GNU tools, as well as the Linux kernel. It can generate cross-platform executables, and is instrumental in getting a new chip, or machine architecture off the ground. Once GCC is self-hosting on the new architecture, many other GNU tools are easy to port to the new architecture.

Currently (as of version 4.5.0), GCC supports the following programming languages:

1. C/C++/Objective-C
2. GNU FORTRAN
3. Java
4. Ada

For the purposes of this section we are not focusing on the internals of GCC, although the interested reader is strongly advised to read the 'GCC Internals' document as well as the source code, as well as refer to Chapter 13 for more information on compiler construction tools and techniques. Instead, we present the common options for GCC command-line, extensions and new features which are of interest in building high-performance, robust applications. It also supports pre-compiled headers, automatic link time instantiation of C++ templates, both features come in handy when compiling large C++ programs.

### *3.1.1 GCC Command-line Options*

The commonly used options for GCC (which are used in almost all non-trivial compilations) are:

- `-v`: print version number of GCC,
- `-I /directory/`: location of additional paths to search for include files present in the program,
- `-L /library/path`: location of additional paths to search for library locations,
- `-l libname`: additionally link to this library also,
- `-o filename`: write output to this filename,
- `-c`: compile only, do not perform linking,
- `-D foo`: define a pre-processor macro on the command-line,
- `-static`: link against static libraries,
- `-fPIC -shared`: link against shared libraries,
- `-ggdb`: include debugging symbols in the output,
- `-On`: perform optimization (using `-O0` switches off optimization),
- `-ansi`: support all ANSI standards,
- `-pedantic -Wall` :turn on warnings,
- `-mtune`: specify machine architecture for optimizations,
- `-fsyntax-only` :quick check of syntax only,

- `-combine`: pass multiple files to compiler at same time,
- `-E`: pre-process only,
- `-S`: produce assemble output,
- `-frepo`: C++ automatic template instantiation at link time,
- `-pipe`: use pipes instead of temporary disk files,
- `-fprofile-arcs`: generate profile data for PGO (profile guided optimization),
- `-fbranch-probabilities`: use PGO runtime data to optimize code,
- `-fomit-frame-pointer`: do not generate stack frames,
- `-pg`: instrument code for profiling,
- `--coverage`: instrument code for code coverage,
- `-fopenmp`: enable handling of OpenMP directives,
- `-fstats`: generate statistics about compilation,
- `-Q`: prints each function name during compilation,
- `-fverbos-asm`: add more text to assembly output,
- `-fWeffc++`: warn about C++ stylistic problems,
- `-fplugin`: load the specified plugin.

GCC also has a number of machine target specific command-line options:

- `-m128bit-long-long`: size of `long long` is 16,
- `-m32`: generate 32-bit code,
- `-m64`: generate 64-bit code,
- `-malign-loops`: align loop code,
- `-mtune`: tune code for a specific architecture.

GCC can also be used in *defensive programming*, by using its options for warning on suspect code. Some options are:

- `-Waddress`: warn about suspicious use of memory addresses,
- `-Wbuiltin-macro-redefined`:
- `-Wwrite-strings`:

Run `gcc --help=warn` for a complete list of command-line options for warning and error generation.

### *3.1.2 GCC Preprocessor*

The preprocessor reads in source code files and performs textual transformations including text replacement, inclusion, conditional expansion and inclusion, before passing the source to the compiler proper. The GNU GCC preprocessor is `cpp` and operates on *directives* placed in the source file. We enumerate the important directives in Table 3.1.

**Table 3.1** Directives in GNU GCC for the preprocessor

| Directive | Description |
|---|---|
| `#include` | inclusion of file into current |
| `#define` | define pre-processor symbol or macro |
| `#undef` | undefine the macro or symbol |
| `#ifdef` | conditional compilation |
| `#else` | based on defined macro |
| `#endif` | -do- |
| `#if` | conditional based on value of variable |
| `#pragma` | additional hints to the compiler |
| `#warning` | generate warning message |
| `#error` | generate error message |
| `#line` | replace current line number |
| `##` | text concatenation operator |

#### 3.1.2.1 GCC Pragmas

The following GCC pragma are useful:

```
#pragma GCC poison malloc
// produces message whenever malloc is used
#pragma GCC dependency "a.inc"
// adds a.inc to dependency list of current file
#pragma GCC system_header
// treats file as a GCC system header
```

#### 3.1.2.2 Predefined Macros

A list of the predefined macros in GCC is given below in Table 3.2.

### *3.1.3 GCC Support of OpenMP*

The OpenMP support in GCC is part of the GOMP project. It is an implementation of OpenMP for GCC supported languages, i.e., C, C++ and GNU FORTRAN. GCC has supported OpenMP since version 4.2. OpenMP code is added to otherwise standard C, C++, FORTRAN code using *pragmas*, or directives to the compiler. The compiler then uses these hints, and the code to produce parallel programs under the semantics of the OpenMP specification. GCC OpenMP uses a runtime library `libgomp` to facilitate the runtime parallelism requirements of the compiled program. The pragmas are written in the form:

```
#pragma omp parallel for
```

**Table 3.2** Directives in GNU GCC for the preprocessor

| Predefined Macro | Description |
|---|---|
| `__BASE_FILE__` | quoted string (full file name) |
| `__cplusplus` | defined when compiling C++ |
| `__DATE__` | current compilation date |
| `__TIME__` | current compilation time |
| `__FILE__` | current translation file |
| `__LINE__` | current line number |
| `__FUNCTION__` | name of function |
| `__PRETTY_FUNCTION__` | for C++ more decorated name |
| `__GNUC__` | major version number of GCC |
| `__GNUC_MINOR__` | minor version of GCC |
| `__VERSION__` | complete version number |
| `__OPTIMIZE__` | defined when optimization is on |
| `__STDC__` | compiler is conformant to C++ |

```
for( i = 0; i < N; ++i )
```

OpenMP is a shared memory model, and thus the user is responsible for variable sharing, and avoiding concurrent access situations, called *race conditions*. Further details about parallel programming with OpenMP and GNU `libgomp` are given in Section 12.2.

### 3.1.4 GCC Advice Mode

Starting with version 4.5.0 GCC has an *advice* mode which gives performance improvement advice for suboptimal usage of C++ data structures and algorithms from the C++ standard library. It is an non-intrusive solution, the application code does not need to be changed, and the advice is call context sensitive. However, to take benefit of this advice mode the application has to be recompiled and executed on representative workloads (akin to profile guided optimization, and branch probability analysis). Consider the following program fragment from a discrete optimization application:

```
  std::ifstream ifs( file_name );
  std::vector< LONG_WORD* > GraphCollection;
  while( ifs ) {
    LONG_WORD* graph = ParseGraph( ifs, dimension );
5   if( graph == NULL ) break;
    GraphCollection.push_back( graph );
    AnalyzeGraph( graph, dimension );
  }
  while( graph_display_mode == 1 ) { // simple printing
10  int which_one = 0;
    std::cerr << std::endl << "Which graph to print ?";
    std::cin >> which_one;
```

```
    if( which_one < 0 ) break;
    LONG_WORD* graph = GraphCollection[ which_one ];
15  ConstructIntoGraph( graph, dimension );
    PrintAdjacencyGraph( which_one, dimension );
  }
```

**Listing 3.1** GCC advice

We compile the full program as follows:

```
g++ -ggdb -D_GLIBCXX_PROFILE -O0 f.cpp -o f_dbg
$./f_dbg <representative input>
cat libstdcxx-profile.txtvector-size: improvement = 2:
 call stack = 0x804abfa 0x94ebb6 0x80499a1 :
 advice = change initial container size from 0 to 108
vector-to-list: improvement = -2:
 call stack = 0x804abfa 0x94ebb6 0x80499a1 :
 advice = change std::vector to std::list
```

Using the `addr2line` program (see Section 13.5.1.1 for more details on using this tool) we can convert the call stack addresses to program line numbers, these correspond to the definition of `GraphCollection` as a `std::vector< LONG_WORD*>`. The representative input we ran on this program to calculate the advice did not include the code in the second `if` clause which uses the data-structure as a random-access container. Thus, GCC's profile based advice informs us that (i) an initial size of 108 for the vector is recommended and (ii) replacing the vector (we only use `push_back`) with a `std::list` (this will actually degrade performance as indicated by the negative score of -2, but can save memory).

### *3.1.5 GCC Attributes*

GCC allows for *attributes* (additional information) to be added to functions and data variables. The attributes are defined using `__attribute__` keyword. There are a number of pre-defined attributes in GCC and these are listed below:

- __noreturn__: specifies that the function does not return to the caller, e.g.,

```
void fatal_error() __attribute__ ((noreturn));
```

- __align__: specifies alignment of data variables, e.g.,

```
struct GraphNode {
   int   value;
   void* data __attribute__ ((align(4)));
};
```

- alias: defines a function to be a weak alias of another function,
- always_inline: if a function has been declared inline, then with this attribute, it will be expanded inline even without any optimization,
- noinline: will never be expanded inline,

- pure: a function with this attribute has no side effects whatsoever (no changes of global variables, but can read global variables),
- const: a *pure* function with the additional constraint that it does not read global variables also (function output is dependent on input parameters only),
- constructor: a function which is called before `main()`,
- destructor: a function called after main() exits,
- format: a function which has a single va_arg style argument,
- malloc: a function which can be treated as `malloc`,
- deprecated: a warning will be issued for every *call*, or place of reference of this variable or function,
- packed: a variable with this attribute has the smallest possible alignment (improves memory footprint of structures).

### *3.1.6 GCC : Inline Assembly*

Inline assembly refers to the ability of the GCC compiler to copy and transform assembly code from the source C/C++ file and insert direct assembly code into the generated assembler output. This facility is useful when optimizing loops, calling CPU instructions which have no equivalent in the language, and calling vector CPU instructions directly. The major differences between writing assembly code versus writing inline assembly are the following:

1. Operand size is determined by the op-code name,
2. Register naming: register names in inline assembly are prefixed by % symbol,
3. variables references: to C/C++ code variables can be made in the inline code.

It should be noted that GCC uses the AT&T assembler syntax as opposed to the Intel x86 syntax. The source-destination ordering in At&T is source, destination. Thus

```
mov %ebx, %eax ; eax <- ebx
movb %al, %bl  ; byte move
movw %ax, %bx  ; word move
movl %eax, %ebx; longword move
movl $0xAABB, %ecx; immediate operand
movb (%eax), %bl; bl <- byte pointed to by eax
```

To write inline assembly, the following syntax has to be observed:

```
asm ( assembler template
    : output operands
    : input operands
    : list of clobbered registers
    );
```

example:

```
#include <stdio.h>
int func(void) {
  int a=1,b=2,c;
  __asm__ ("movl %1, %%eax\n\t"
   "movl %2, %%ebx\n\t"
   "add %%eax,%%ebx\n\t"
   "movl %%ebx,%0\n\t"
   : "=r"(c)
   : "r"(a),"r"(b)
   : "%eax","%ebx"
   );
  return c;
}

int func2(void) {
  int a=1,b=2,c;
  __asm__ ("add %1,%2\n\t"
   "movl %2,%0\n\t"
   : "=r"(c)
   : "r"(a),"r"(b)
   : /* none */
   );
  return c;
}

int main( ) {
  printf("%d %d", func(), func2() );
  return 0;
}
```

The relevant part of the assembler output is shown below:

```
# func
# 4 "inl.c" 1
movl %edx, %eax
movl %ecx, %ebx
add %eax,%ebx
movl %ebx,%edx
# func2
# 17 "inl.c" 1
add %eax,%edx
movl %edx,%eax
```

The critical difference between `func` and `func2` is that, in func2 we did not specify to GCC which registers are to be used for variables `a` and `b`. In register strapped architectures such as x86, long instruction sequences increase the *register pressure*. Unless a specific register is required by the instruction (such as CLD), it is

better to let GCC choose a register. In this case (when GCC automatically chooses a register), that register need not be added to the list of clobbered registers.

#### 3.1.6.1 cpuid instruction

Above we had mentioned the use of inline assembly in calling CPU instructions which do not have a C/C++ language equivalent. One such instruction is the `cpuid` instruction which returns information about the current CPU. An example of using inline assembly with this instruction is shown:

```
/* CPUID instruction,
   calling with 0 returns the maximum call value

*/
#include <stdio.h>
int get_max_cpuid_call(void) {
  int retval = 0;
  __asm__ ("xor %%eax, %%eax\n\t"
   "cpuid\n\t"
   "movl %%eax,%0\n\t"
   : "=r"(retval)
   : /* no inputs */
   : "%eax"
   );
  return retval;
}

int print_ids(void) {
  int id;
  int ax;
  for( ax=0; ax < get_max_cpuid_call(); ++ax ) {
    __asm__ ( "movl %1, %%eax\n\t"
      "cpuid\n\t"
      "movl %%eax,%0\n\t"
      : "=r"(id)
      : "r"(ax)
      : "%eax" );
    printf("ID[%d] = %d\n", ax, id );
  }
}
#define cpuid(func,ax,bx,cx,dx)  \
  __asm__ __volatile__ ("cpuid": \
 "=a" (ax), "=b" (bx), "=c" (cx), "=d" (dx) : "a" (func));



unsigned int get_cpu_information(void) {
  unsigned int info = 0x0;
  int param = 1;
  __asm__ ( "movl $1, %%eax\n\t"
    "cpuid\n\t"
    "movl %%eax,%0\n\t"
```

```
    : "=r"(info)
    : "r"(param)
    : "%eax" );
  return info;
}
```

```
void print_cpu_information(unsigned int param) {
  printf("\n CPU Information = %d", param);
  printf("\n Stepping number = %d", (param & 7) );
  printf("\n Model number = %d", (param & 0x00F0) >> 4 );
  printf("\n Family number = %d",(param & 0x0F00) >> 8 );
  printf("\n Processor type= %d",(param & 0xA000) >> 12);
}

int main() {
  printf("Max CPUID call = %d\n", get_max_cpuid_call() );
  print_ids();
  print_cpu_information( get_cpu_information() );
  printf("\n");
  return 0;
}
```

The output of the previous program is shown below:

```
Max CPUID call = 2
ID[0] = 2
ID[1] = 1750

 CPU Information = 1750
 Stepping number = 6
 Model number = 13
 Family number = 6
 Processor type= 0
```

One inherent problem with inline assembly is that, not only is the code now specific to GCC (or atleast to a compiler which understands GCC style inline assembly), but more importantly, the code is now specific to an instruction set. Thus, it is a good idea to encapsulate inline assembly code in functions and for equivalent C/C++ code (if it exists) to be maintained at the same time as assembly. The results of the C/C++ code should be compared against the inline assembly code as well. In the next section we discuss GCC intrinsics which in some cases remove the necessity of using inline assembly, while maintaining some portability.

### *3.1.7 GCC Intrinsics*

GNU GCC contains a number of *extensions* to the C/C++ language, not the least of which are the *intrinsic* functions. Intrinsics are optimized versions of functions which have been tuned for a particular architecture or instruction set. The extensions supported by GCC include:

- `typeof`: returns the type-of (akin to sizeof) an expression,

- long long: double-word integer,
- __int128: 128-bit integers,
- Local labels: labels which are local to a block,
- Statement expressions: are permitted with this extension,
- Nested functions: are permitted with this extension,
- Intrinsic functions: discussed below,
- Empty structures: are allowed,
- Thread local variables: using the __thread storage class,
- Return address: getting the return address from within the function,
- Offset of: calculates the offset of a particular data member within a structure.

#### 3.1.7.1 X86 builtin functions: intrinsics

The major categories of builtins are:

1. Mathematical functions: such as sin, cos,
2. Compilation hints: include functions __builtin_types_compatible_p, __builtin_expect, __builtin_constant_p, __builtin_prefetch,
3. Atomic functions: these functions implement *atomic* operation functions. They include, fetch-and-add, compare-and-swap, synchronize, and lock-and-test. These functions are full *barrier* functions, thus no memory operand will be moved across the operation. Atomic builtins can be used to implement *spin-locks* and other synchronization facilities,
4. Vector arithmetic: SIMD on small size vectors, e.g., `fabsq`, `padd`, and `psub`,
5. Memory load/store, shuffle instructions:
6. CRC32: checksum functions,
7. AES: encryption functions,
8. Bit counting: population count for number of bits set.

### *3.1.8 Compiling Java using GCC*

We can use the `gcj` GNU Java compiler to compile Java programs. Consider the pedantic and canonical Java program:

```
/* helloworld.java */
public class helloworld {
    public static void main( String arg[] ) {
        System.out.println( "Hello!\n" );
    }
}
```

We can compile this program as:

```
gcj --main=helloworld helloworld.java -o H
```

One key distinction between using GNU `gcj` and other Java compilers such as `javac`, is that GNU `gcj` produces native compiled code binaries, which do not need to be run on the Java Virtual Machine (JVM). This has advantages of performance, but at the same time the compile once, run everywhere facility of Java is discarded.

### 3.1.9 Compiling Ada using GCC

Consider the simple hello, world style program in Ada.

```
-- this is hello world in Ada
with Text_IO; use Text_IO;
procedure HelloWorld is
begin
   Put_Line( "Hello from Ada!" );
end HelloWorld;
```

We create a file `helloworld.adb` with these content, then we can compile this Ada program using the GNU gnat system.

```
$gnatmake helloworld.adb
gcc -c helloworld.adb
gnatbind -x helloworld.ali
gnatlink helloworld.ali
$./helloworld
Hello from Ada!
```

### 3.1.10 Conclusion

Most of the open-source software developed on GNU/Linux is compiled using some version of GCC, and thus it is an essential part of the open-source software stack. Using GCC effectively is important, and in this section we have described the important facets of using GCC and its related tools. In particular, GCC extensions, pragmas, support for OpenMP, attributes, inline assembly, and builtin intrinsics were discussed. Using GCC to compile languages other than C/C++ was also briefly discussed.

## 3.2 Source Code Configuration Systems

### *3.2.1 Introduction to Version Control Systems*

Software configuration management (SCM) and version control system (VCS) are used to manage changes to program text, documents and other information stored in computer files. It is used during the process of writing and developing computer software, where a number of people working on the same software product are developing and working on the same set of files. Using SCM and VCS software, the programmers aim to collaborate on the development, while maintaining the ability to work on private copies of the shared source code files. Changes are usually referred to as a version number and are a combination of numeric version numbers and letters.

As the team continues to develop software it is common to introduce new features, and fix bugs existing in previous versions of the software. Since the life of the deployed software is many years, the software development team must be able to go back to a *snapshot* of the released version to reproduce a customer bug, or fix a problem. Moreover, during development, errors can creep in, and developers can use the version control system to do a quick analysis of the change they have made to the code files from the last known good version.

Another common feature of the version control system is the concept of *branches*, where the files at some point in time are *forked* into differing versions, presumably for experimental development, or feature freeze prior to release. We shall present the example of the software systems to support branching below. Once the feature under development has been completed, it is possible to *merge* the changes back to the main development line, which is often called the *trunk*, to take the analogy of the software development tree further.

Moder software development systems and integrated development environments (IDEs) have built in features to access version control systems through the user interface. In this chapter we discuss the common version control systems including CVS, SVN, and git. We discuss the command line utilities to access the version control repository, and also discuss some graphical tools.

### *3.2.2 CVS*

As mentioned above CVS (concurrent version system) is a version control system. The main activities for working with a CVS *repository* can be divided into two roles (i) the repository administrator, and (ii) CVS user. CVS allows the following features:

To begin working with CVS, first the location of the CVS *repository* has to be informed to the CVS tools. A common method is to define the CVSROOT environ-

ment variable with the location of the repository. If the repository is a local file it can be specified as:

```
export CVSROOT=/usr/local/cvsroot
```

or when working with multiple repositories it can be specified on the command-line for every CVS tool as:

```
cvs -d /usr/local/cvsroot diff file.c
```

Once a local copy of the repository has been checked out, CVS maintains the current CVSROOT information in the file CVS/Root. It is possible to connect to a remote repository using the RSH protocol.

- cvs init: this command creates a repository in the named location with access controls. The choice of the machine and file system to deploy CVS depends on the expected load on the repository server, concurrent usage, and work load.
- cvs import: given an existing collection of files and directories, these can be added to the CVS repository using the import command. An example is:

  ```
  cvs import -m "Import" POLY POLY init
  ```

  this commands add all the files and directories in the current working directory as a child directory and files in the POLY directory of the repository. The commit log is given by the -m argument, and the init tag is specified.
- cvs checkout: copies a copy of the module or file specified on the command line. Internally, CVS also maintains hidden directories which contain the repository information for each directory. Example:

  ```
  $cvs co POLY
  cvs checkout: Updating POLY
  U POLY/cano_proc.cpp
  U POLY/dual.cpp
  U POLY/graph_reader.cpp
  ```

  the directory structure within POLY/CVS is shown below:

  ```
  drwxrwxr-x. 2  4096 2010-06-13 17:52 .
  drwxrwxr-x. 3  4096 2010-06-13 17:52 ..
  -rw-rw-r--. 1   150 2010-06-13 17:52 Entries
  -rw-rw-r--. 1     5 2010-06-13 17:52 Repository
  -rw-rw-r--. 1    24 2010-06-13 17:52 Root
  ```

- cvs update: periodically update the local copy of the files to bring in changes from the repository.

  ```
  RCS file: /home/skoranne/CVSROOT/POLY/cano_proc.cpp,v
  retrieving revision 1.1.1.1
  retrieving revision 1.2
  Merging differences between 1.1.1.1 and 1.2
                                  into cano_proc.cpp
  ```

```
rcsmerge: warning: conflicts during merge
cvs update: conflicts found in cano_proc.cpp
C cano_proc.cpp
```

The conflict markers show the difference between the local changes and the modifications made on the repository copy since the last update.

```
static void WritePostamble( std::ofstream& ofs ) {
<<<<<<< cano_proc.cpp
        // this function should have some comments
=======
        // I also want to add some comments
>>>>>>> 1.2
  ofs << std::endl << "\\end{document}" << std::endl;
}
```

- cvs commit: once changes in the file have been made, to make them apparent to other users of the CVS system, the changes have to be committed back to the repository. This is don't using the cvs commit command; this command requires an accompanying log for each commit. The log can be specified on the command line using the -m option, or an editor can be invoked.

```
$cvs commit -m "Add my comments" cano_proc.cpp
Checking in cano_proc.cpp;
CVSROOT/POLY/cano_proc.cpp,v  <--  cano_proc.cpp
new revision: 1.2; previous revision: 1.1
done
```

- cvs diff: this command runs the `diff` command on the current version of the file with the CVS copy. Results are presented in the `diff` format.

```
[skoranne@celex POLY]$ cvs diff cano_proc.cpp
Index: cano_proc.cpp
===================================================
RCS file: /home/skoranne/CVSROOT/POLY/cano_proc.cpp,v
retrieving revision 1.1.1.1
diff -r1.1.1.1 cano_proc.cpp
130a131
>  // I also want to add some comments
```

- cvs stat: shows the status of the current files:

```
===================================================
File: cano_proc.cpp     Status: Locally Modified

   Working revision: 1.1.1.1 Mon Jun 14 00:50:22 2010
   Repository revision: 1.1.1.1 CVSROOT/POLY/cano_proc.cpp,v
   Sticky Tag: (none)
   Sticky Date: (none)
   Sticky Options: (none)
```

- cvs release: removes file from the working copy, if specified using -d option physically deletes the file from disk as well.

A full list of CVS commands is:

```
add          Add a new file/directory
admin        Administration front end for rcs
annotate     Show last revision
checkout     Checkout sources for editing
commit       Check files into the repository
diff         Show differences between revisions
edit         Get ready to edit a watched file
editors      See who is editing a watched file
export       Export sources from CVS
history      Show repository access history
import       Import sources into CVS
init         Create a CVS repository if it doesn't exist
log          Print out history information for files
login        Prompt for password for authenticating server
logout       Removes entry in .cvspass for remote repository
pserver      Password server mode
release      Indicate that a Module is no longer in use
remove       Remove an entry from the repository
rtag         Add a symbolic tag to a module
server       Server mode
status       Display status information
tag          Add a symbolic tag
unedit       Undo an edit command
update       Bring work tree in sync
version      Show current CVS version(s)
watch        Set watches
watchers     See who is watching a file
```

As an alternative to memorizing the CVS command, the graphical tool TkCVS can be used as shown in Subsection 3.2.5.

### *3.2.3 SVN*

Subversion, or SVN is designed to be a better version control system than CVS, but for ease of migration it has retained many of the same command-line functions as CVS. In addition it has the following features:

- Commits as true atomic operations,
- Renamed/copied/moved/removed files retain full revision history,
- The system maintains versioning for directories, renames, and file metadata,
- Branching and tagging as cheap operations,
- Costs proportional to change size,
- XML log output,
- File locking for unmergeable files,
- Path-based authorization.

Running `svn stat` on a locally checked out revision gives:

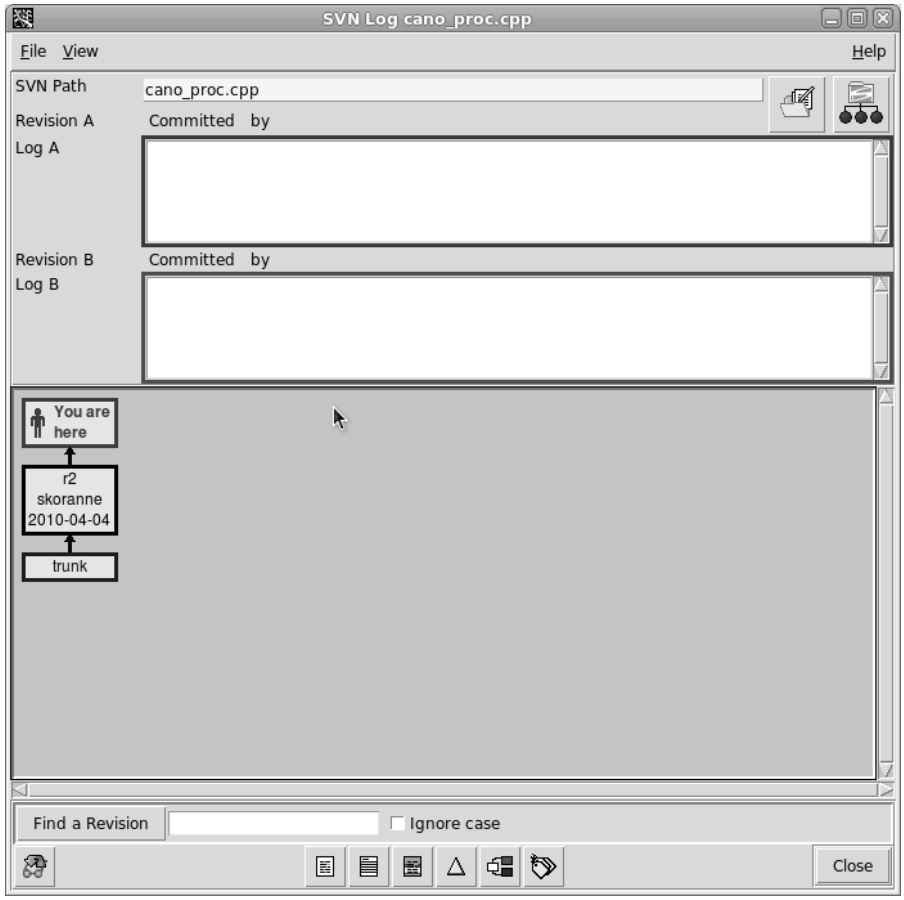



**Fig. 3.1** TkCVS showing log of file

```
?       build.sh
M       graph_reader.cpp
```

The '?' symbol shows that the file 'build.sh' is not in the version control, while the file graph_reader.cpp has been locally modified. We can inspect the changes to the file using the `svn diff` command or by using the TkCVS graphical tool.

```
--- graph_reader.cpp (revision 2)
+++ graph_reader.cpp (working copy)
@@ -96,8 +96,10 @@
  GRAPH[i][j] : GRAPH[i][k]+GRAPH[k][j];
```

Some of the available commands for SVN are given below, by design many of the commands have similar name and purpose as the above mentioned CVS commands.

```
add
blame (praise, annotate, ann)
changelist (cl)
checkout (co)
cleanup
commit (ci)
```

```
copy (cp)
delete (del, remove, rm)
diff (di)
export
import
merge
move (mv, rename, ren)
revert
status (stat, st)
update (up)
```

### 3.2.4 GIT

Git is a distributed revision control system with an emphasis on speed. Git was initially designed and developed by Linus Torvalds for Linux kernel development. Every Git working directory is a full-fledged repository with complete history and full revision tracking capabilities, not dependent on network access or a central server.

### 3.2.5 TkCVS

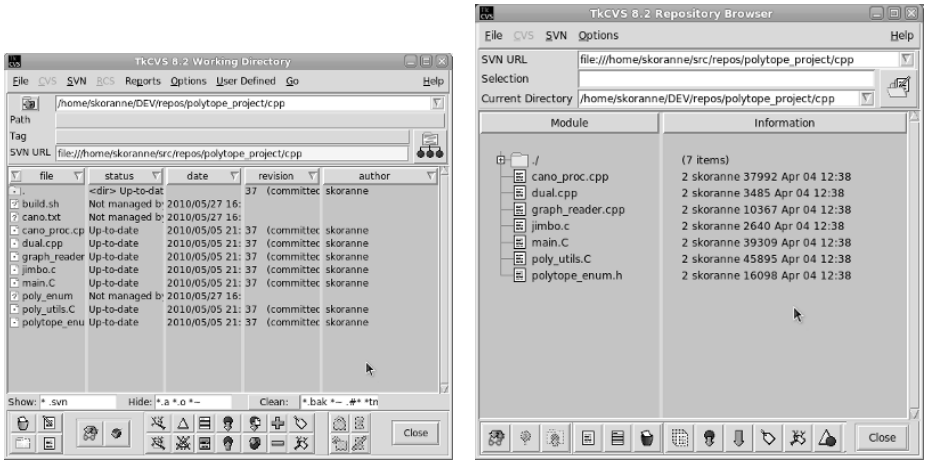
TkCVS is a graphical tool which makes it easy to work with diverse version control systems. It can support CVS, SVN, and other version control systems. TkCVS is a Tcl/Tk-based graphical interface to the CVS and Subversion configuration management systems. It displays the status of the files in the current working directory, and provides buttons and menus to execute configuration-management commands on the selected files. Limited RCS functionality is also present. TkDiff is bundled in for browsing and merging your changes.

TkCVS also aids in browsing the repository. For Subversion, the repository tree is browsed like an ordinary file tree. For CVS, the CVSROOT/modules file is read. TkCVS extends CVS with a method to produce a browsable listing of modules. TkCVS is invoked as:

```
tkcvs [-dir directory] [-root cvsroot]
      [-win workdir|module|merge] [-log file]
```

### 3.2.6 Tinderbox

Tinderbox is a software suite that provides continuous integration capability. Tinderbox allows developers to manage software builds and to correlate build failures on various platforms and configurations with particular code changes.




(a) TkCVS showing current branch (b) TkCVS showing repository model

**Fig. 3.2** TkCVS branch and repository

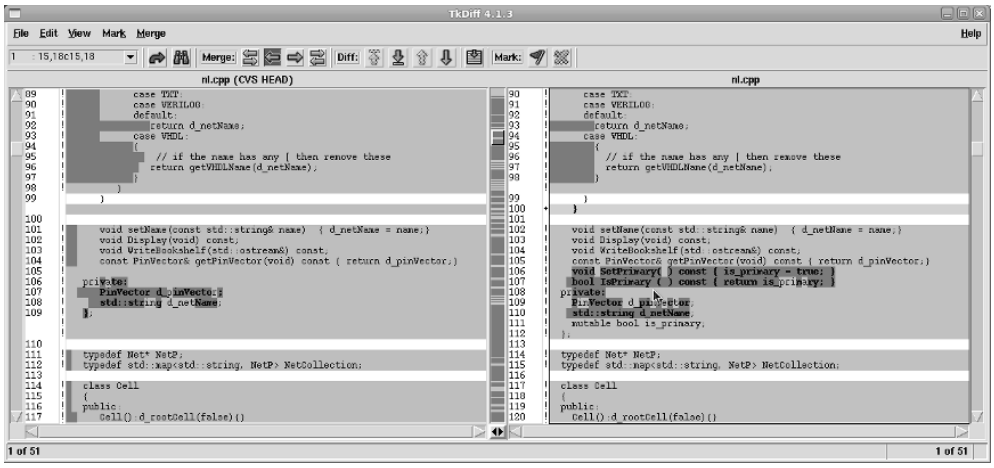



**Fig. 3.3** TkCVS showing TkDiff module with CVS

Tinderbox is described as “a detective tool for software development”. It allows the developer to see what is happening in the source tree. It shows who checked in what (by asking Bonsai); what platforms have built successfully; what platforms are broken and exactly how they are broken (the build logs); and the state of the files that made up the build (cvsblame). Tinderbox is composed of a server with clients running builds and reporting status via mail.

## 3.3 GNU Build System

As the reader probably knows already, most GNU software has a traditional way of installation: (i) download the source TAR file, (ii) untar the archive in a local directory, and (iii) run:

```
$./configure
$gmake
$gmake install
```

The goal of the GNU build system is to simplify the development of portable programs, as well as encourage the open-source concept by simplifying the process of releasing products as source code which can then be compiled by the user. Moreover, using the GNU build tools allows for the VPATH build concept where the source code is placed in a single directory while operating-system or machine dependent variations of the build are carried out in their own directories. In this section we focus our attention on the `configure` script which is omnipresent in GNU software. This script is generated as a part of the GNU Build Tools process which involves the following tools (i) autoconf, (ii) automake, and (iii) libtool; these are described below.

### *3.3.1 Autoconf*

`autoconf` produces a configuration shell script (`configure`) which probes the host system for portability related information. This information is then used as part of the build, but can also be used by the application program. The most common use of `autoconf` is to generate customized source code blocks using pre-processor `#define` on the computed values.

The version of `autoconf` can be checked using:

```
$autoconf --version
autoconf (GNU Autoconf) 2.63
```

### *3.3.2 Automake*

`automake` produces make files (see Section 3.4.1 for more information on GNU `gmake`). The tool produces the file Makefile.in to be used by `autoconf` from a very highlevel specification file called Makefile.am, thus reducing the effort required.

We use a simple example to describe the use of `autoconf` and `automake`. Before running the GNU build tools the directory containing the example is:

```
$ ls
```

```
configure.in  hello_world.cpp  Makefile.am
```

The files 'configure.in' and 'Makefile.am' are shown below:

```
# \file configure.in
# \author Sandeep Koranne, (C) 2010
# \description Example file for autoconf

AC_INIT(hello_world.cpp)
AM_INIT_AUTOMAKE(hello_world,1.0)
AC_PROG_CXX
AC_PROG_INSTALL
AC_OUTPUT(Makefile)
```

The statements listed in the 'configure.in' file above initialize the configuration system (AC_INIT), any source code file name can passed as an argument. The AM_INIT_AUTOMAKE denotes the use of `automake` and also contains the name of the package and version number. Features on the host system, such as version of C++ compiler can be checked using AC_PROG_CXX statement. To check for `lex` (see Section 13.3) we can add AC_PROG_LEX, and AC_PROG_YACC (for `yacc`).

```
# \file Makefile.am
# \author Sandeep Koranne, (C) 2010
# \description Example file for automake

bin_PROGRAMS = hello_world
hello_world_SOURCES = hello_world.cpp
```

We proceed to run `autoconf` as:

```
$aclocal
$ls
aclocal.m4  autom4te.cache  configure.in
hello_world.cpp  Makefile.am
$autoconf
$ ls
aclocal.m4      configure     hello_world.cpp
autom4te.cache  configure.in  Makefile.am
```

Finally, we have generated our own '.configure' file, its contents being too numerous to copy here. Next we run `automake -A` to get:

```
$automake -a
configure.in:6: installing `./install-sh'
configure.in:6: installing `./missing'
Makefile.am: installing `./INSTALL'
Makefile.am: required file `./NEWS' not found
Makefile.am: required file `./README' not found
Makefile.am: required file `./AUTHORS' not found
```

```
Makefile.am: required file `./ChangeLog' not found
Makefile.am: installing `./COPYING' using GNU
General Public License v3 file
$ ls
aclocal.m4      configure     COPYING
hello_world.cpp  install-sh   missing
autom4te.cache  configure.in  depcomp
INSTALL          Makefile.am
```

To create the required files 'NEWS', 'README', we can create them in an editor, or simply call `touch NEWS` to create an empty file which we promise to fill with useful and relevant data *later on*. Thereafter, we have to run `automake -a` again, and this time it completes without any error, creating the file 'Makefile.in' which is used by 'configure'. Running './configure' gets:

```
$ ./configure
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /bin/mkdir -p
checking for gawk... gawk
checking whether make sets $(MAKE)... yes
checking for g++... g++
checking for C++ compiler default output file name... a.out
checking whether the C++ compiler works... yes
checking whether we are cross compiling... no
checking for suffix of executables...
checking for suffix of object files... o
checking whether we are using the GNU C++ compiler... yes
checking whether g++ accepts -g... yes
checking for style of include used by make... GNU
checking dependency style of g++... gcc3
checking for a BSD-compatible install... /usr/bin/install -c
configure: creating ./config.status
config.status: creating Makefile
config.status: executing depfiles commands
```

This produces the Makefile which can be processed using GNU `gmake` as:

```
gmake
g++ -DPACKAGE_NAME=\"\" -DPACKAGE_TARNAME=\"\"
    -DPACKAGE_VERSION=\"\" -DPACKAGE_STRING=\"\"
    -DPACKAGE_BUGREPORT=\"\" -DPACKAGE=\"hello_world\"
    -DVERSION=\"1.0\" -I. -g -O2 -MT hello_world.o
    -MD -MP -MF .deps/hello_world.Tpo
    -c -o hello_world.o hello_world.cpp
mv -f .deps/hello_world.Tpo .deps/hello_world.Po
g++  -g -O2   -o hello_world hello_world.o
```

The produced Makefile conforms to the GNU Makefile standard and has the default targets of 'clean', 'distclean'. An additional target 'distcheck' produces a

TAR gzipped archive of the current sources and proceeds to test it against the listed regression suite.

### *3.3.3 Libtool*

The `libtool` tool is a platform independent method of generating shared libraries and position independent code. It supports versioning of shared libraries. The `libtool` command-line tool can be used independently of `automake` and `autoconf`, but is used by them for compiling position independent code and generating shared libraries.

The above example may appear contrived and trivial to warrant the use of GNU Build tools, and indeed it is, as it is an expository and pedantic example. However, for large and complex software which is expected to be compiled on many different systems (some of which have not even been invented yet), the separation between source code and operating system and machine dependencies is very important. The GNU Build tools help enforce this separation, and indeed the tools are used in most open-source applications, thus even if the reader does not have to write a Makefile.am file from scratch, at the very least its purpose and semantics should be understood.

## 3.4 Automatic Build Dependency Management

We have already seen example of building source code using the GNU Build tools. One clear advantage of using automated build tools is the reduction of effort in making a new binary from source code changes. However, a naive approach to recompiling the whole software suite since a single file has been modified does not scale well beyond small projects. Towards that end, automated build dependency tools have been developed. In this section we discuss two tools for automated build dependency, (i) GNU `gmake` and (ii) SCons.

### *3.4.1 GNU make : automatic build dependency*

GNU Make (`gmake`) is a utility for automatically building executable programs and libraries from source code. An input control file (called a Makefile) specifies the dependencies of the various source files (using gcc -MD, the dependencies based on inclusion of files can be generated automatically). The `make` tool then computes the topological sort of the files and performs the required actions on the dependencies to produce the *target*.

The control file specifies the relationship between the target and the dependencies. A simple rule looks like:

```
target:     dependencies
            action
```

If the time stamp of the source dependency is more recent than the time stamp of the target, the target needs to be *rebuilt*. The procedure for the rebuilding entails the execution of the commands in the control file. Consider an example where a static library depends on hundreds of object files (each of which is a compiled C/C++ file); in turn the library contributes to a binary. This can be represented as:

```
libapp.a:    parser.o optimizer.o app.o
      $(AR) cr libapp.a parser.o optimizer.o app.o
app.exe:     libapp.a main.o
      $(CC) $(CFALGS) main.c -lapp -o app.exe
```

When specifying rules and actions GNU make understands the following features:

- Variables: variables can be defined in the Makefile using

  ```
  APP_INCLUDE := -I$(apps)
  ```

  predefined variables for the current rule are:

  ```
  $< : current list of depencies which changed
  $@ : current target
  $^ : all dependencies of current rule
  ```

  There are also targets which can be marked as `.PHONY` implying that they do not represent real on-disk files, so that even the presence of a real file of that name does not match the target.
- Pattern substitution:

  ```
  OBJS    = $(patsubst %.cpp,%.o,$(SOURCES))
  ```

  or pattern substitution based on extension with pre-defined actions:

  ```
  .cpp.o:
         $(CXX) $(CXXFLAGS) -c $< -o $@
  ```

- Include files: using `include` to bring in parts of a global Makefile (containing rules or dependencies),
- Conditionals: GNU make supports conditional dependencies, consider

  ```
  ifdef STATIC
  CCOPT += -static
  endif
  ```

  Then on the command-line we can say `gmake STATIC=1` to enable the variable. The second form of the conditional is based on the value of variables:

```
ifeq ($(cc), gcc )
     $(CC) -mcpu=pentium -c file.c
else
     $(CC) -c file.c
```

In the above example, if the variable `cc` has a value `gcc` then we compile the source file with the `-mcpu` command line option.

- Functions for file names and others: GNU make has inbuilt functions which operate on file names. These functions include:

```
$(dir src/file.c ) : produces src
$(suffix src/file.c src/app.h) : produces .c .h
$(basename src/file.c) : produces src/file
$(join a b, .c .o) : produces a.c b.o
$(wildcard src/*.c): produces wildcard expansion
$(foreach ) example
   files := $(foreach dir,$(dirs),$(wildcard $(dir)/*))
$(call variable,param, param):
$(eval ) function
```

In addition to the Makefile syntax there are a number of commonly used command-line options to gmake:

```
-B : unconditionally make all targets
-k : continue despite errors
-d : print lots of debugging information
-f : specify name of Makefile
-n : dry-run, dont perform any action
```

### *3.4.2 SCONS : A software construction tool*

SCons is a computer software construction tool that automatically analyzes source code file dependencies. Its function is analogous to the traditional GNU build system based on the make utility and the autoconf tools. SCons uses the Python general purpose programming language as a foundation, so that all software project configurations and build process implementations are Python scripts.

Consider a simple program in C/C++ (hw.cpp):

```
#include <iostream>
int main( ) {
  std::cout << "Hello, World!\n";
  return 0;
}
```

The SCons build file for this application is extremely simple: we create a file called `SConstruct` and in it we add

```
$cat SConstruct
Program('hw.cpp')
$scons
scons: Reading SConscript files ...
scons: done reading SConscript files.
scons: Building targets ...
g++ -o hw.o -c hw.cpp
g++ -o hw hw.o
scons: done building targets.
$./hw
Hello, World!
```

To build an object file, instead of `Program` use the `Object` *builder method*, and for building libraries use the Library builder. The SCons tool knows the default actions it needs to perform for the given builder object. For Program it compiled and linked the file, for Object it only performs the compilation. The Program construct can also take the name of the produced binary as the first argument, as:

```
Program( 'new_hello', 'hello.c' )
```

Multiple files can be passed as a list to the Python builder object, or by using a wildcard matching `Glob(*.c)`. Multiple Program builder objects can be placed in a single file to build more than one program. Since SCons file are Python scripts, and the Program objects are Python objects, their constructors can accept named parameters; to link a program with a library we can specify it as:

```
Library('compression', ['lz77.c', 'file_io.c'])
Program('main.c', LIBS=['compression'], LIBPATH='.')
```

#### 3.4.2.1 The SConstruct file

SConstruct files are not like normal Python programs, in that they are not executed sequentially. It is after-all a Makefile equivalent, and the order in which the builder objects are *defined* in the SConstruct file are independent from the order in which the targets are built.

**Node objects** Builder methods return a list of node objects that identify the target file. These node objects themselves can be passed as arguments to other builder methods. Moreover, properties on node objects can be used to control various build options for specific files and targets.

**SCons dependencies** Just as `GNU make` can be used to calculate the minimum set of files which have to be rebuilt to build a target (make uses timestamp information to populate the change graph), SCons also calculates the set of files to be rebuilt. Instead of using the timestamp, SCons uses a MD5 *signature* or checksum for each file (although SCons can be configured to use timestamp). It is even possible to specify a custom Python function to decide if an input file has changed (imagine a tool where CVS/SVN changes on the repository are automatically inspected to calculate

change information for some files). SCons MD5 mechanism is robust and supports include files and symbolic links.

Calculating which files have changed is just part of the dependency analysis. The main graph of dependencies has to be constructed as well. For this purpose, SCons uses the CPPPATH variable for every source file. For every include file of the source file located in the CPPPATH directory, SCons adds a dependency from the included file to the source file. If the included file has been modified, the source file has to be recompiled.

For Library builder object a dependency is added from every member of the list to the library itself. Scanning each source file takes extra time, but as the contents of the file are being inspected for the MD5 anyway, the include information can be cached as well. This is performed by using the implicit cache command-line discussed below.

To explicitly add dependencies, we can use the Depends('A','B') method. Additionally, we can use GCC's -MD option to generate the dependencies as:

```
A = Object('file.c',
           CCFLAGS='-MD -MF file.d',
           CPPPATH='.')
SideEffect('file.d',A)
ParseDepends('file.d')
```

Dependencies (for a specific target) can be ignored using the Ignore method.

**SCons Environments** An *environment* defines a related set of option to be used to build a piece of the software. An environment can be created using:

```
env = Environment()
```

method. An easy method to create a custom build environment based on operating system environment variables is:

```
import os
env = Environment(CXX = 'g++', CXXFLAGS = '-ggdb')
env.Program('file.cpp')
```

This SConstruct file specifies that file.cpp source should be compiled using the debug option unless the CXXFLAGS variable is defined to some other value in the OS. The default environment can be set using the DefaultEnvironment function.

In addition to the control file SCONS has a number of command-line options:

```
-c : remove specified targets
-Q : reduce verbosity of messages
--implicit-cache : cache dependencies
--random : build dependencies in random order
```

A more detailed example of using SCons is shown in Listing 3.2.

```
import os
env = DefaultEnvironment()
print "Default CXX = ", env['CXX']

dbg = Environment(CXXFLAGS='-O0 -ggdb')
```

```
opt = Environment(CXXFLAGS='-O3 -funroll-loops')

if ARGUMENTS.get('opt', 0):
    env = opt
else:
    env = dbg

env.Program('udp_server.cpp', LIBS=['boost_system','pthread'],
            CPPPATH=['/home/skoranne/INCLUDE'],
            LIBPATH=['/usr/lib'])
Program('udp_client.cpp', LIBS=['boost_system','pthread'])
```

**Listing 3.2** SCons file for Boost UDP

## *3.4.3 CMAKE and QMake*

### 3.4.3.1 CMake

CMake is a cross platform, open-source make system. CMake generates native Makefiles (which can be processed with GNU make). Nokia/Trolltech Qt's `qmake` also generates Makefile for the platform. Like SCons, CMake maintains the control file in a named file, in this case CMakeLists.txt; an example is given below:

```
project(app)

CMAKE_MINIMUM_REQUIRED(VERSION 2.4.5 FATAL_ERROR)
if ( GCC_FOUND )

add_config_flag( CGAL_USE_GMP )
add_config_flag( CGAL_USE_MPFR )

endif()
macro( add_programs subdir target ON_OFF )

  cache_set( CGAL_EXECUTABLE_TARGETS "" )

endmacro()
```

Run `man cmake` for more information on CMake.

### 3.4.3.2 QMake

QMake is Qt's build program. An example qmake '.pro' file is shown below:

```
TEMPLATE = app
TARGET =
DEPENDPATH += .
```

```
INCLUDEPATH += .

# Input
SOURCES += qsig.cpp
```

Unlike SCons or CMake, qmake .pro (for project) file has a simpler keyword-value syntax. In addition to turning off Qt libraries, it also allows for specifying debug or release builds, location of Qt resources such as bitmaps, and additional libraries. Like CMake, qmake also generates platform specific Makefiles, which are then processed using GNU make to build the target.

## 3.5 Bugzilla : Defect Tracking System

Bugzilla is a defect tracking system implemented as a server software featuring optimized database structure for efficiency, security, advanced query tools, and integrated email. Bugzilla supports the complete life-cycle of a defect, from ticket initiation to final resolution. Moreover, it can be used to generate charts and reports with diverse statistics on the defects stored in its database.

After installation, the initial Bugzilla screen in the browser is shown in Figure 3.4.



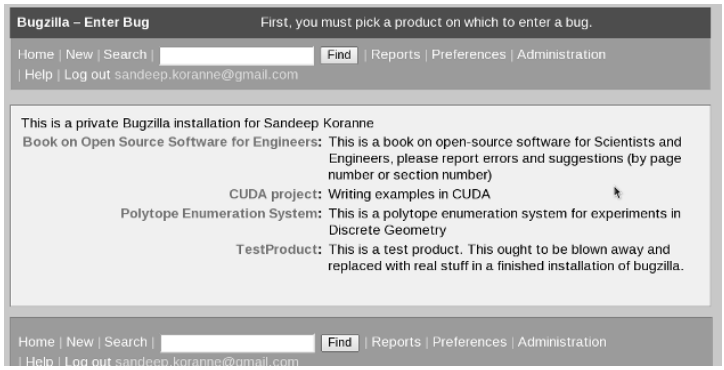

**Fig. 3.4** Bugzilla screen

To deploy Bugzilla the system must have:

1. compatible database management system
2. suitable release of Perl 5
3. assortment of Perl modules
4. compatible web server
5. suitable mail transfer agent, or any SMTP server

These requirements are fairly easy to meet on a modern GNU/Linux system, and deploying Bugzilla is easy, see Figure 3.4 for the main Bugzilla screen where user

authentication can be performed. This book maintained its own Bugzilla database during its authoring. We have added *products* defining the scope of the book material and divided the products into *components* as shown in Figure 3.5.




(a) Adding new product to Bugzilla




(b) Filing a bug

**Fig. 3.5** Setting up and using Bugzilla

## 3.6 Editing Source Code

Many disciplines (if not most) of science now routinely use and develop software as part of the research. Software development systems have been designed to aid the software engineering process, of which writing the original code is an integral part. All software starts with some source code written in a file or storage system. In this section we discuss open-source code editors

### *3.6.1 Emacs*

Emacs (acronym for Editor Macros) is an extensible text editor, however, its use goes much further than simple editing of code and text. Emacs has more than 1,000 commands for editing, and also allows the user to write new functions using Emacs Lisp, and to attach these new functions to keyboard shortcuts. Thus, the user can define keyboard macros to automate repetitive tasks (such as running LaTeXon a section to preview it). An example of using Emacs to edit source code is shown in Figure 3.6.

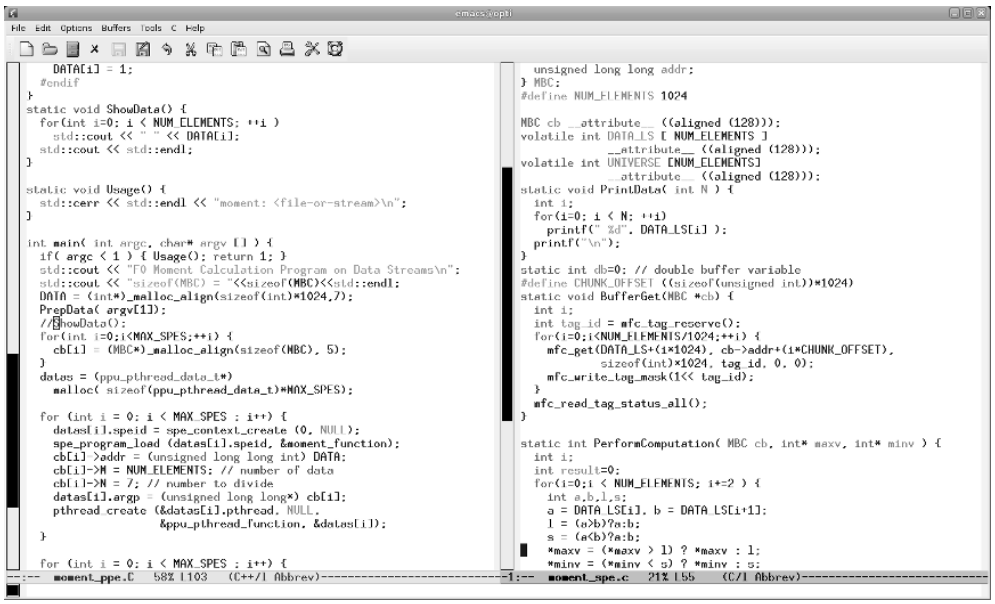



**Fig. 3.6** EMACS: editor for code editing

As shown in Figure 3.6, Emacs can be used as a syntax highlighting source code editor. Using Emacs Lisp, various *modes* have been written which automate tasks such as indentation, highlighting and even using the C++ compiler to check for source code errors while typing.

Emacs can be used effectively as an IDE (integrated development environment) as it can read TAGS (see Section 3.7.1) data, as well as having customized interaction capability for source-code editing. Using the `speedbar` command, relevant information on files in the current directory of the editor can be inspected, as shown in Figure 3.7.

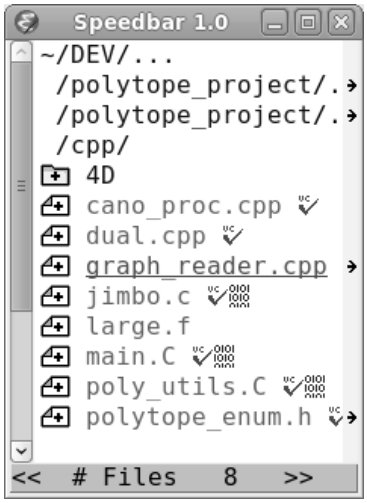



**Fig. 3.7** 'speedbar' in Emacs

Emacs also has an interactive front-end to GDB, as well as hooks to popular version control systems such as CVS (see Section 3.2.2) and Subversion (see Section 3.2.3). File merging, difference checking, and annotations can also be performed within Emacs.

### *3.6.2 Eclipse*

Some of the tools and development environments which are integrated with Eclipse are listed below:

- CDT: C and C++ development,
- PTP: Parallel Tools Platform which contains a scalable parallel debugger,
- Cell Broadband Development: environment for developing software on the Cell Broadband Engine,

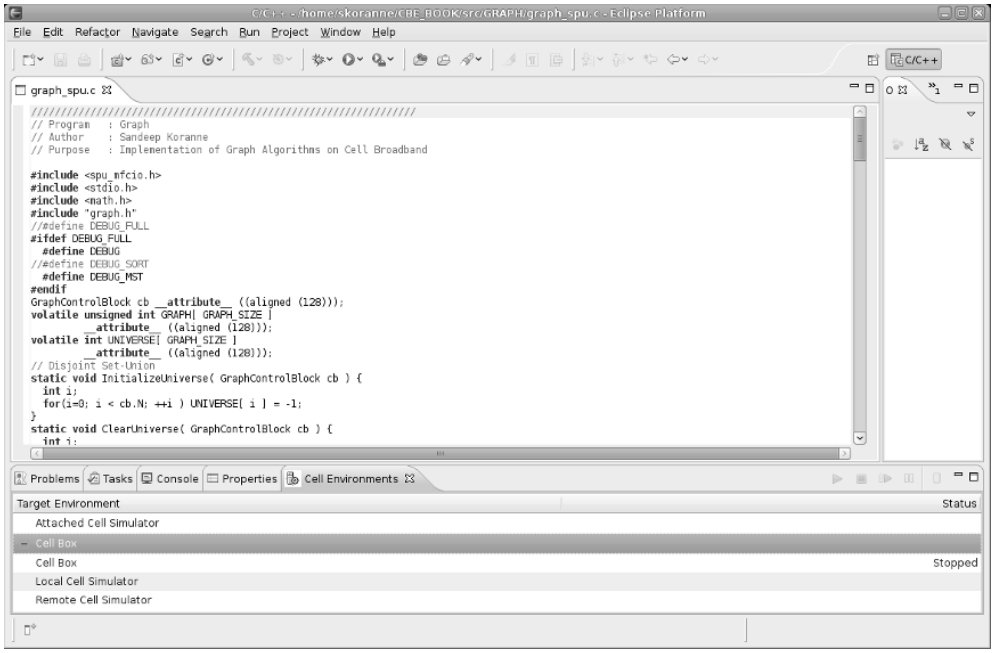



**Fig. 3.8** ECLIPSE: editor for code editing

### *3.6.3 KDevelop*

KDevelop is a free software integrated development environment for the KDE desktop environment for Unix-like computer operating systems. See Figure 3.9 for an example of a session running within kdevelop.

Its features include:

1. Source code editor with syntax highlighting and automatic indentation,
2. Project management for different project types using automake, qmake,
3. Class browser,
4. Integrated debugger (using gdb),
5. Automatic code completion (C/C++), and class definitions,
6. SCM support for CVS, SVN, Perforce.

KDevelop's dialog for creating a new project is shown in Figure 3.10.

The settings for the project can be changed within KDevelop, as shown in Figure 3.11.

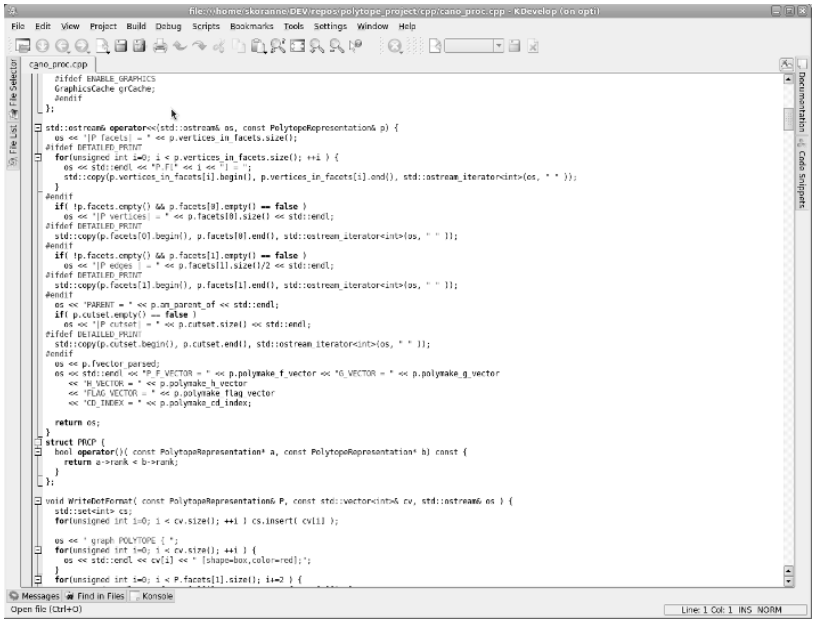


**Fig. 3.9** KDEVELOP: IDE for code editing

**Fig. 3.10** KDEVELOP: Dialog for creating new project

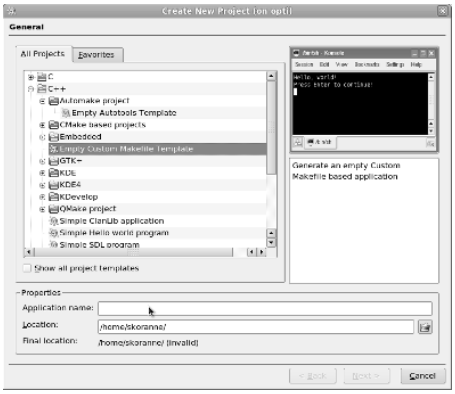
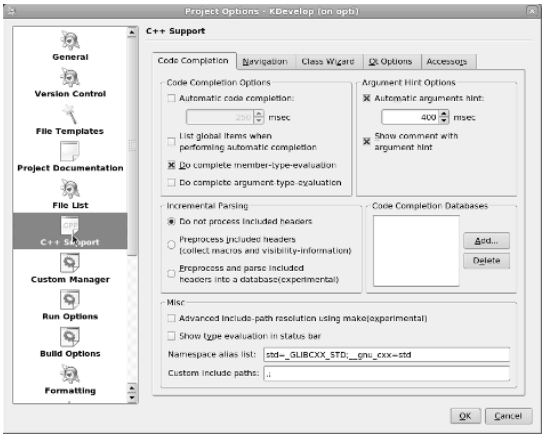


**Fig. 3.11** KDEVELOP: Dialog for project options



## 3.7 Static Checks on Source Code

In addition to run-time checks (profiling, debugging) another class of checks on source code which is very helpful in reducing errors are *static checks*. Static checking involves analysis of source code. In this section we discuss two tools which can provide analysis on source code.

### *3.7.1 ctags*

The `ctags` and `etags` programs generate index file for a number of programming language files. The index file contains a listing of the various source-code objects detected in the input files. This index file can thereafter be used by source-code editors to quickly locate a section of source-code text which contains an object, such as variable name, or function definition.

The `ctags` program has a number of command-line options which control its output. It can be run in the root directory of the source code tree with the `-R` (recursive) option to generate an index for all known source code files located under the root. It can also *append* TAGS to a file. Consider the following example of `ctags` being run to produce Emacs (see Section 3.6.1) compliant TAGS index files.

```
$ctags -e -R --verbose=yes|grep OPEN
OPENING dual.cpp as C++ language file
OPENING build.sh as Sh language file
OPENING large.f as Fortran language file
OPENING jimbo.c as C language file
OPENING poly_utils.C as C++ language file
OPENING enum.scons as Python language file
OPENING graph_reader.cpp as C++ language file
```

A part of the produced TAGS file (which is a binary file) is also shown; it can be seen that `ctags` has encoded information about source-code objects in the file.

```
dual.cpp,464
typedef std::vector<int> Vector;^?Vector^A37,653
```

In Emacs, using `tags-apropos` we can perform a regular-expression search for any symbol present in the TAGS file:

```
Tags matching regexp 'polytope.*hash':

[poly_utils.C]: std::set< std::string > GlobalPolytopeHash;
[poly_utils.C]: bool MarkUniquePolytope( const CBPolytope&
[poly_utils.C]: string& CBPolytope::compute_hash_code( ) const {
[cano_proc.cpp]:   PolytopeRepresentation():hash_code(""),
```

Using the `-x` option to `ctags` causes it to produce a tabular, human-readable cross-reference file instead of the TAGS file. Example:

```
LONG_WORD   function 19 gr_reader.cpp LONG_WORD():j1(0x0) {}
LONG_WORD   struct   17 gr_reader.cpp struct LONG_WORD {
MAX_DIM     macro    29 gr_reader.cpp #define MAX_DIM 10
MAX_N       macro    58 gr_reader.cpp #define MAX_N 128
MY_INT_MAX  macro    64 gr_reader.cpp #define MY_INT_MAX 10000
```

The cross-reference file contains the tag-name, type, location in source-code. This file can be post-processed to produce statistical data about the source-code. The use of `ctags` is not limited to source-code for programming, it can also be used to generate TAGS file for other types of text documents, as `ctags` supports many languages, including Bash scripts, and even TEXand LATEX.

## 3.8 GNU gcov: Test Coverage Program

GNU `gcov` is a test coverage program. It is used in conjunction with GNU `gcc` to analyze the program and to uncover untested parts of the source code. The `gcov` tool can be used to analyze which part of the source code is being executed to better aid and guide algorithmic optimization efforts. It can also be used to discover portions of the source code which are not covered by the program testbench. When used alongwith a profiling tool such as `gprof` (see Section 3.12.1), `gcov` can be used to calculate source code metrics such as (i) execution count of each line of source code, (ii) which source code lines are actually executed, and (iii) compute time spent in each section of code.

In this section we describe the use of `gcov` to calculate the test coverage of a program. We assume that a testbench for the program exists and is representative of the common usage expected of the program. Ofcourse, `gcov` is also used to guide the development of the said testcase, as it can be used to pinpoint source code areas which are un-stressed. To use `gcov` effectively, the program should be compiled without optimization and with debug symbols in place. This can be achieved using the following command-line for GNU `gcc`:

```
$gcc -O0 -ggdb -c <filename>
```

Moreover, since `gcov` computes statistics on a line-by-line basis, it works best with programs which have only one statement per source code line. Similarly, the use of complex macros causes `gcov` to report line usage after macro expansion, which can produce confusing results. A complex macro should be wrapped in an inline function for this purpose.

### *3.8.1 Compiling programs for gcov*

To use `gcov` the source must be compiled to report the execution count and branch probabilities. This is referred to as *instrumenting* the source code, and is similar to

the action of compiling with the -pg flag for profiling (see Section 3.12.1). The GNU gcc command-line options for compiling with gcov are:

```
$gcc -fprofile-arcs -ftest-coverage <file> -lgcov
```

The -lgcov argument *links* the GNU gcov library (which is required for the code instrumentation to work) with the application.

### 3.8.2 Running gcov

gcov has a number of command-line options as listed below:

-a --all-blocks: write individual execution counts for every block,
-c --branch-counts: write branch frequencies as number of branches taken rather than percentage,
-b --branch-probabilities: write branch frequencies to the output file, and write branch summary information to standard output,
-f --function-summaries: output summaries for each function in addition to file summary,

Consider the small program shown in Listing 3.3.

```
// \file gcov_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using GNU gcov program
#include <iostream>           // for program IO
#include <cassert>            // for assertion checking
#include <cstdlib>            // for exit

static const unsigned int N = 16;

static int gMatrix[N][N];

static void InitMatrix(void) {
  for( unsigned int i=0; i < N; ++i )
    for( unsigned int j=0; j < N; ++j )
      gMatrix[i][j] = i + j;
}

int main( int argc, char *argv[] ) {

  std::cout << "Example of using GNU gcov.\n";

  return (0);
}
```

**Listing 3.3** Example of using gcov

We compile this program as:

```
g++ -ggdb -O0 -fprofile-generate -fprofile-arcs \
              -ftest-coverage gcov_example.cpp
$ls
a.out  gcov_example.cpp  gcov_example.gcno
```

We then run the program on representative input:

```
$./a.out
$ls *.gcda
gcov_example.gcda
```

The execution of the program produces a binary data file `.gcda`, which contains the execution count, branch probabilities, and other information about the program's execution. We can now use `gcov` to analyze this file:

```
$gcov -a -b -c -l -f -p -u gcov_example.cpp
Function '_ZStorSt12_Ios_IostateS_'
Lines executed:0.00% of 2
No branches
No calls

Function '_ZL10InitMatrixv'
Lines executed:0.00% of 5
No branches
No calls

Function '_ZNKSt9basic_iosIcSt11char_traitsIcEE7rdstateEv'
Lines executed:0.00% of 2
No branches
No calls
....
Function 'main'
Lines executed:100.00% of 3
No branches
No calls
```

In addition to the report generated on standard output, `gcov` also creates detailed reports for each of the *translation units* and header files it encounters in the source file. C++ language header files are also annotated, though for the most part (for application developers) execution counts within these system files can be ignored.

What is important though, is the reported execution count per line for the application program source code, as that directly corresponds to the testability of the program. If a particular section of code has very low execution count, it means that, the code in question has not been exercised enough in the testbench. The generated annotation for our original code (from Listing 3.3) is shown in Listing 3.4.

```
        -:    0:Source:gcov_example.cpp
        -:    0:Graph:gcov_example.gcno
        -:    0:Data:gcov_example.gcda
        -:    0:Runs:1
    5   -:    0:Programs:1
        -:    1:// \file gcov_example.cpp
        -:    2:// \author Sandeep Koranne, (C) 2010
        -:    3:// \description Example of using GNU gcov program
        -:    4:#include <iostream>           // for program IO
   10   -:    5:#include <cassert>            // for assertion checking
        -:    6:#include <cstdlib>            // for exit
        -:    7:
        -:    8:static const unsigned int N = 16;
        -:    9:
   15   -:   10:static int gMatrix[N][N];
```

```
        -:   11:
    #####:   12:static void InitMatrix(void) {
    #####:   13:  for( unsigned int i=0; i < N; ++i )
    +++++:   13-block  0
    +++++:   13-block  1
    +++++:   13-block  2
    #####:   14:    for( unsigned int j=0; j < N; ++j )
    +++++:   14-block  0
    +++++:   14-block  1
    +++++:   14-block  2
    #####:   15:      gMatrix[i][j] = i + j;
    #####:   16:}
    +++++:   16-block  0
        -:   17:
        1:   18:int main( int argc, char *argv[] ) {
        -:   19:
        1:   20:  std::cout << "Example of using GNU gcov.\n";
        1:   20-block  0
        -:   21:
        1:   22:  return (0);
        3:   23:}
        1:   23-block  0
        1:   23-block  1
        1:   23-block  2
        1:   23-block  3
```

**Listing 3.4** gcov generated annotated source code

Since we did not call the function InitMatrix at all in the file, execution count for the section of code in that function is 0, and the whole section is marked as unused code. Calling the function, recompiling and re-running the code fixes this problem.

Branch probabilities as calculated by -fprofile-arcs are also used in *feedback directed optimization* in GNU gcc. With the deep pipeline present in current generation CPUs, such optimization is very effective and can easily reduce runtime by 15 to 20 percent.

When we run gcov -a to get information for all the blocks we see in the generated .gcov file the following type of information:

```
    -:   11:
#####:   12:static void InitMatrix(void) {
#####:   13:  for( unsigned int i=0; i < N; ++i )
$$$$$:   13-block  0
$$$$$:   13-block  1
$$$$$:   13-block  2
#####:   14:    for( unsigned int j=0; j < N; ++j )
$$$$$:   14-block  0
```

On another code block which has heavy branching, running gcov -b produced:

```
  1117:  586: if(A_set & B_set) assert(A.empty()==false);
branch  0 taken 28% (fallthrough)
branch  1 taken 72%
call    2 returned 100%
branch  3 taken 0% (fallthrough)
branch  4 taken 100%
call    5 never executed
  1117:  587: if( A.empty()) assert((A_set&B_set)==0);
call    0 returned 100%
```

```
branch  1 taken 72% (fallthrough)
branch  2 taken 28%
branch  3 taken 0% (fallthrough)
branch  4 taken 100%
call    5 never executed
```

We can use the branch counts to optimize the code by introducing GCC's intrinsic functions for branch hints; see Section 3.1.7.1. We can add

```
#if defined (__GNUC__) && ( __GNUC__ > 2 ) && defined(__OPTIMIZE__)
#define PROB1(expr) (__builtin_expect((expr),1))
#define PROB0(expr) (__builtin_expect((expr),0))
#else
#define PROB1(expr) (expr)
#define PROB0(expr) (expr)
#endif
```

Then, whenever we have a condition for which we know the branch probability with high certainty, we can code it as:

```
if( PROB0( error_case ) ) {
   std::cerr << ``Unlikely, but has to be handled...\n'';
}
```

We performed a simple experiment using this technique on a compute intensive workload of discrete geometry. We ran the unmodified program first, then we added PROB0 and PROB1 to a total of 5 conditional branch points (all branch points were within loops), and we recompiled and reran the program on same input. The results are shown below:

```
Before optimization

45.97user 1.96system 0:53.49elapsed 89%CPU
0inputs+7304outputs (0major+20828minor)pagefaults 0swaps

After optimization
41.82user 1.84system 0:46.24elapsed 94%CPU
0inputs+7304outputs (0major+20841minor)pagefaults 0swaps
```

Ofcourse, this is a very small runtime example, but over longer running programs, even a 10-15 percentage improvement is substantial.

It should be noted that the execution counts present in the '.gcda' files are cumulative over multiple runs of the program (this is useful in test coverage analysis where a collection of tests is expected to test the whole program).

## 3.9 Debug Tools

### *3.9.1 GDB*

In this section we describe the GNU debugger, `gdb`. Almost all non-trivial programs go through a process of finding bugs (see Section 3.8 on code coverage), and this procedure is called *debugging*. A good software debugger can reduce the time to find problems in the code by an order of magnitude. The GNU debugger is called `gdb`, and it has the following command-line options:

```
gdb [options] [exe [core-file or pid] ]
```

- –cd = dir: change current directory to dir,
- –core=COREFILE: analyze the core dump file,
- –directory=dir: search dir for source code,
- –exec=EXECFILE: use EXECFILE as the executable binary,
- –command=FILE, -x: execute gdb commands from file,
- –version: print version and exit.

The main commands of `gdb` can be found using `help` command, we list the important and often used commands below, their usage is described later in this section.

aliases aliases of other commands
breakpoints making program stop at certain points
data examining data
files specifying files
running Running the program
stack Examining the stack

Consider the program listing shown in Listing 3.5.

```
// \file number_test.cpp
// \author Sandeep Koranne, (C) 2010
// \description C++ example for GDB
// Fibonacci = (0,1,1,2,3,5,8,13,21,34,55,89)
// Sum = prefix-sum of F
// (0,1,2,4,7,12,20,
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
/**
   \function SumFibonacci
   @param s0, s1 contain initial fibonacci numbers
   @param N represents how many numbers to sum
   @return sum of N fibo numbers from (s0,s1)
   \description Algorithm is non-recursive
 */
unsigned int SumFibonacci( int s0, int s1, int N ) {
  int i,j;
  unsigned int next=s0+s1;
  unsigned int sum=0;
  for(i=0; i < N; ++i ) {
    next = s0 + s1;
```

```
    s1 = next;
    s0 = s1;
    sum += next;
  }
  return sum ;
}

int main( int argc, char * argv[] ) {
  int s0=0, s1=1;
  int i,j,rc=0;
  unsigned int sum = 0;
  for( i=3; i < 8; ++i ) {
    sum = SumFibonacci( 0, 1, i );
    printf("Sum = %d\n", sum );
  }
  return ( EXIT_SUCCESS );
}
```

**Listing 3.5** Example for debugging with GDB

Listing 3.5 computes the sub of the first $N$ Fibonacci numbers (Fibonacci numbers are defined by the recurrence $F_n = F_{n-1} + F_{n-2}$. We compile this program and run it. We expect the sequence to be $(1,2,4,7,12,20,$, but there is atleast one error in the program and we turn to `gdb` to find it and fix it.

We compile the program using `-ggdb` option to GCC to produce a binary with debug symbols. We also turn off optimization using the `-O0` option. We then run `gdb number_test` to run GDB:

```
(gdb) b main
Breakpoint 1 at 0x80484d3: file number_test.cpp, line 31.
(gdb) list
32   int i,j,rc=0;
33   unsigned int sum = 0;
34   for( i=3; i < 8; ++i ) {
```

We also put a *breakpoint* on the function `SumFibonacci`. We can use TAB-completion to query GDB's loaded symbol table from the binary to complete the function name.

```
(gdb) b SumFibonacci(int, int, int)
Breakpoint 2 at 0x804847a: file number_test.cpp, line 19.
```

We then start execution of the program by issuing the `run` command to GDB; any command line arguments to our program can be added to the `run` command. The program starts to execute, and hits our first break point `main`. We then proceed to *step* through the program using the `gdb step` command, which is aliased to 's'; the command `next` is aliased to 'n'. Since we know the problem is inside the `SumFibonacci` function (as the inputs to the function are trivial), and we have a breakpoint inside the function we can *continue* using the **`continue`** command, aliased to `conti`. By debugging the function we can see that the for-loop is incorrect, and that the order of the assignments of `s0,s1` is incorrect.

The current stack (sequence of functions in last in first out) order is displayed using `where`.

```
(gdb) where
#0  SumFibonacci (s0=0, s1=1, N=3) at number_test.cpp:23
#1  0x08048519 in main (argc=1, argv=0xbfffec54) at
    number_test.cpp:35
(gdb)
```

To display the value of local and global variables we can use the `display` and `print` functions. We can also dump the content of the binary, a memory location, and variables to a file. We can inspect the value of CPU registers, and if the program under debug is multi-threaded we can switch between threads. If the program was compiled with debug symbols the current code at the program counter of the currently debugged thread can be displayed using the `list` command:

```
(gdb) list
18   int i,j;
19   unsigned int next=s0+s1;
20   unsigned int sum=0;
21   for(i=0; i < N; ++i ) {
22
```

We can quit GDB using the `quit` command, aliased to 'q'. GNU gdb also has remote debugging capabilities where it can connect to a remote system running the binary, while the source code resides on the host system. It can also connect to a running process, and analyze crash dump files.

GNU `gdb` is a versatile tool and recent versions have added significant features, and more are expected in the future (such as running the program backwards to debug an error condition). Although GNU `gdb` itself is a command-line driven text tool, several frontends and GUIs have been developed around it, and it has been integrated in IDEs such as Eclipse (see Section 3.6.2) and Kdevelop (see Section 3.6.3. Emacs has a frontend for GDB called `gud`, see more details on Emacs as an IDE in Section 3.6.1.

A frontend for GNU `gdb` is Insight, and we describe it in the next section.

### *3.9.2 Insight*

As mentioned above, Insight is a frontend of GNU `gdb`. We have already discussed the command-line usage in Section 3.9.1, and in this section we discuss a graphical frontend to `gdb`. Launching Insight on a binary which has debug symbol, we get the source window with the `main` function as shown in Figure 3.12.

In addition to the GUI controls, the existing `gdb` style commands also work; we can enter them in the console window as shown in Figure 3.13.

In GNU `gdb` we can display the content of registers and memory locations but in Insight there is a graphical display which shows data-structures as unions and classes, as shown in Figure 3.14(a) and (b); registers with their contents are shown in Figure 3.15.

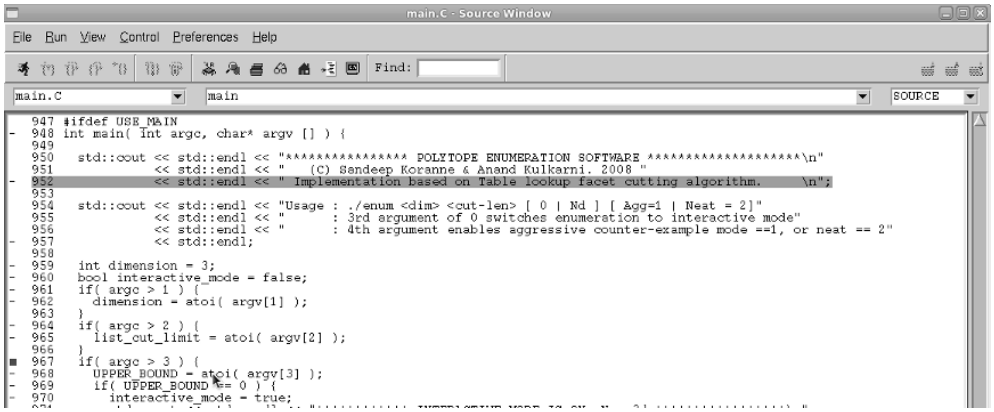



**Fig. 3.12** Insight : gdb front end as debugger

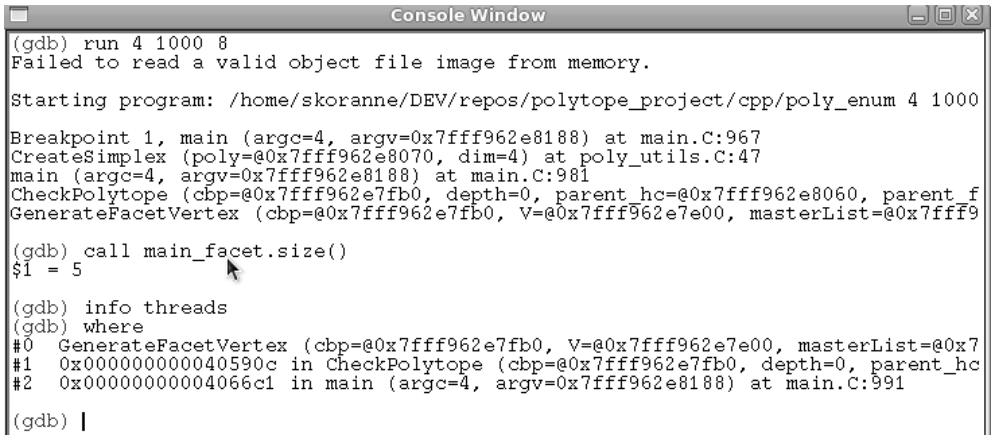
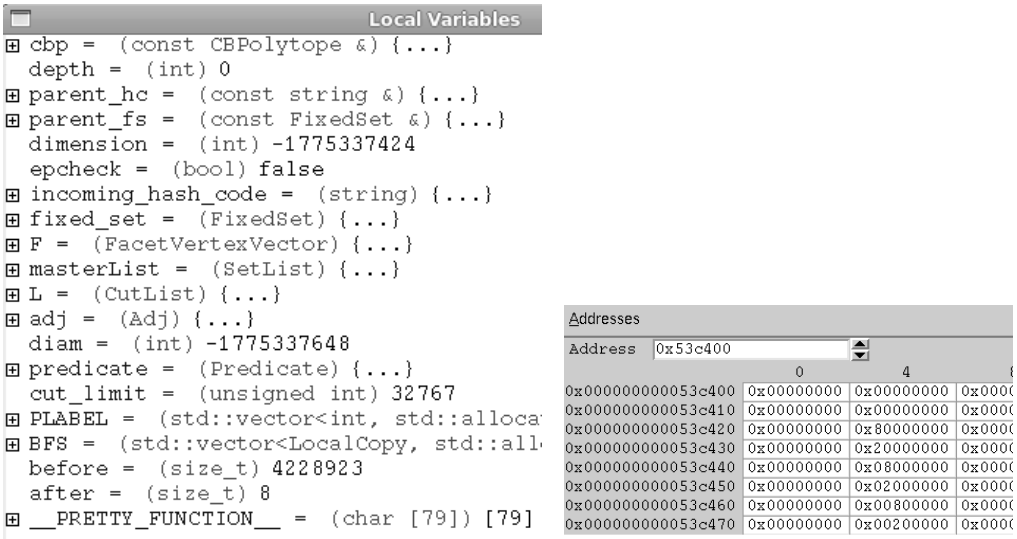



**Fig. 3.13** Insight console front end for gdb

## 3.10 Doxygen

Doxygen is an automated document generator for source code written in C/C++, Java and many other programming languages. It is a document processor and can generate output in many formats including HTML, RTF, and LaTeX(which can then be converted to PS, PDF). Doxygen uses a control file which specifies the location of the source code and options for output generation. By default it generates HTML from source code, but the source code itself can be used to *markup* aspects of the program which are of particular interest, such as main algorithms, invariants, parameters passed to functions. For object oriented languages, Doxygen can generate class diagrams using the `dot` graph drawing tool (see Section 19.4). Doxygen is used for generating, maintaining and writing reference documentation for software. It is a valuable tool in refactoring, or understanding a complex project.




(a) Local variables and memory (b) Memory dump

**Fig. 3.14** More examples of debugging using Insight

.

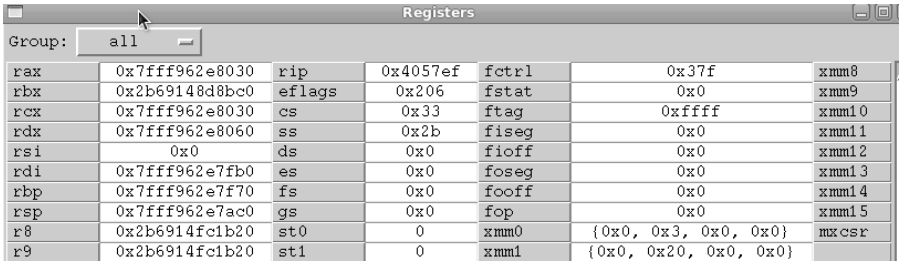



**Fig. 3.15** Insight: register view and memory watch

.

## *3.10.1 Using Doxygen*

Doxygen itself can be used to generate a template configuration file using the `doxygen -g` command-line option. An example config file is shown below.

### 3.10.1.1 Writing .doxy files

```
DOXYFILE_ENCODING      = UTF-8
PROJECT_NAME           = polytope
PROJECT_NUMBER         = 1.0
OUTPUT_DIRECTORY       = Doutput
INPUT                  = cpp lisp python
FILE_PATTERNS          = *.cpp *.h *.c *.py *.lisp
```

```
CLASS_GRAPH            = YES
COLLABORATION_GRAPH    = YES
GROUP_GRAPHS           = YES
INCLUDE_GRAPH          = YES
INCLUDED_BY_GRAPH      = YES
CALL_GRAPH             = YES
DIRECTORY_GRAPH        = YES
DOT_IMAGE_FORMAT       = png
DOT_CLEANUP            = YES
```

#### 3.10.1.2 Generating output

Output can now be generated using `doxygen Doxyfile` command. Doxygen generates HTML which can be opened using any Web browser, we show an example in Figure 3.16.

#### 3.10.1.3 Doxygen markup in source code

The quality of the generated document from Doxygen can be enhanced if the author of the source code adds additional *markup* to the source code (using special comments and tags). We describe some of the source code markup for Doxygen below:

1. @file: name of the file,
2. @author: name of the author,
3. @version: version of the file,
4. @section: part of the source code (or algorithm),
5. @class: name of the class,
6. @param: parameter passed to function,
7. @return: return value from function.

Special comments using `///`, and `//!` can also be used to generate additional information in the Doxygen output.

## 3.11 Source Navigation

SourceNavigator is a source code analysis tool. It can be used to edit source, manage source code projects, display dependencies between classes and functions, and display call trees. An example of a project managed by SourceNavigator is shown in Figure 3.17(a) and Figure 3.17(b).

When importing an existing project into SourceNavigator, the project can be automatically converted to its native format. To create a new project, individual source code directories can also be added. The SourceNavigator code analysis engine then

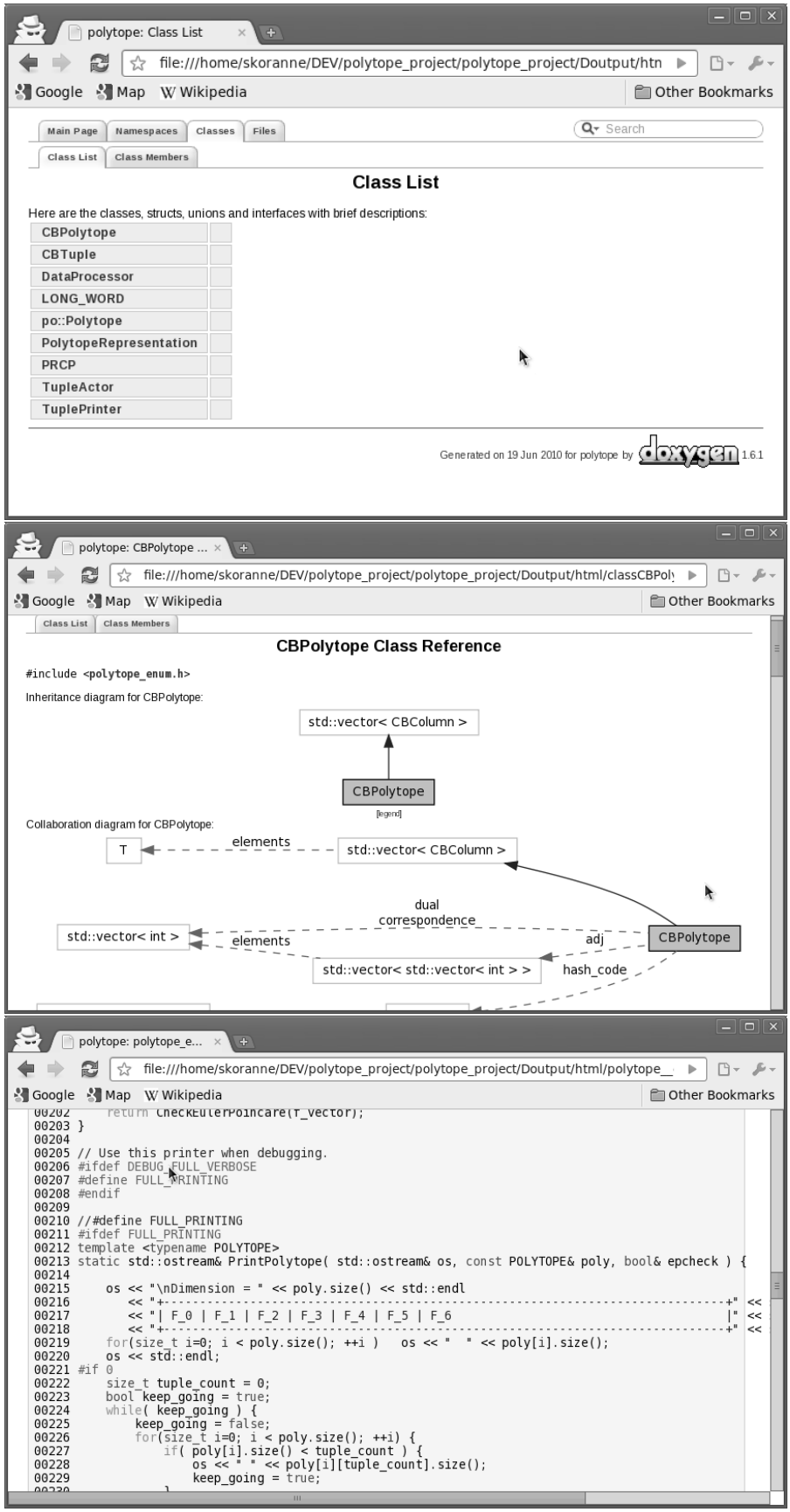
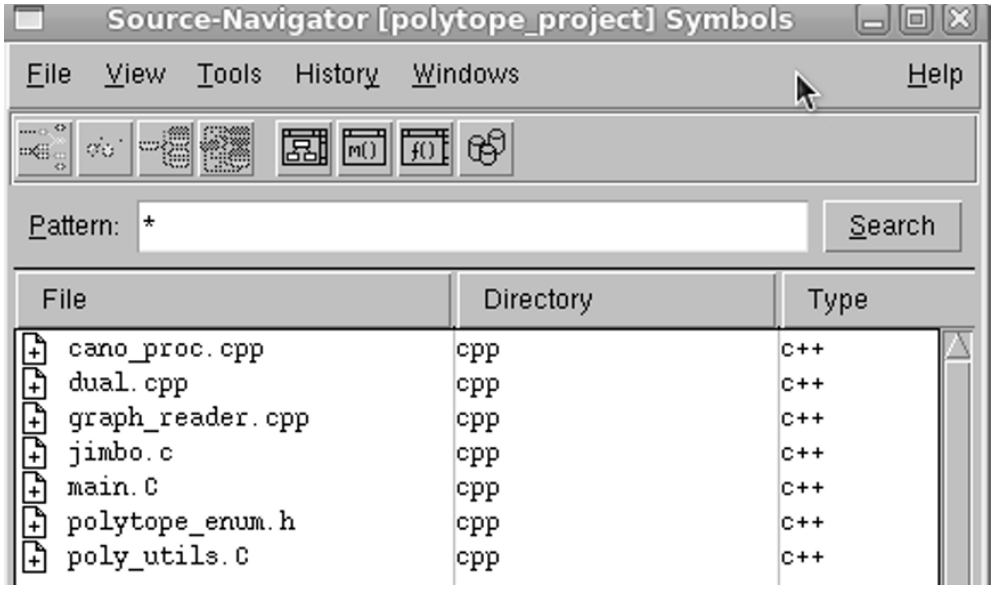
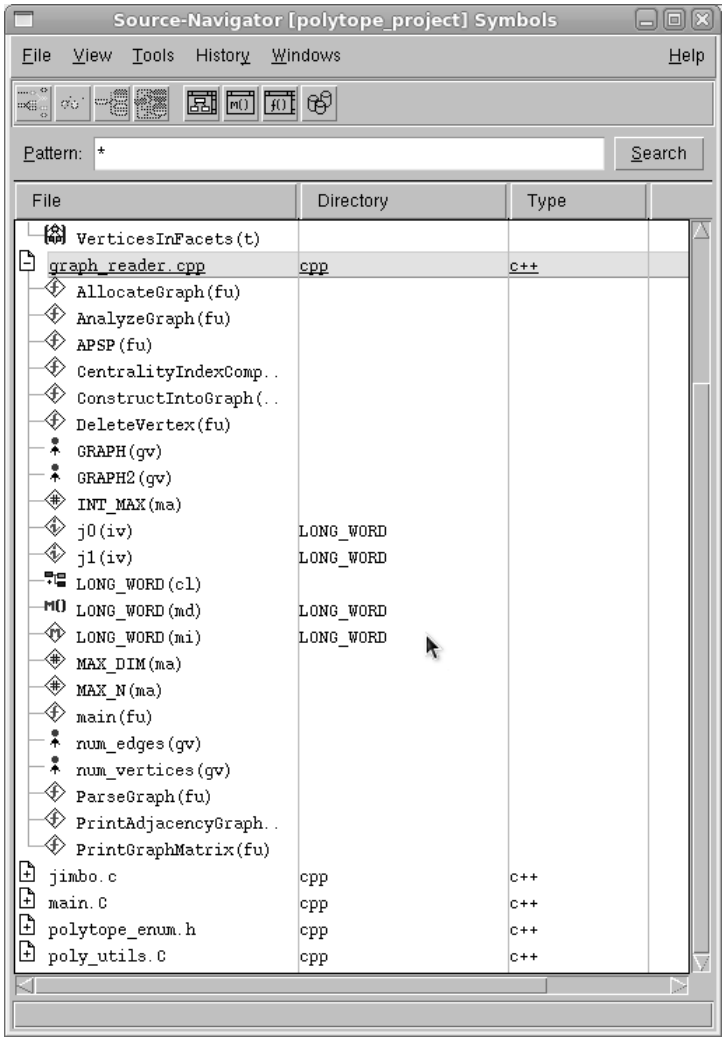



**Fig. 3.16** Doxygen : document generator

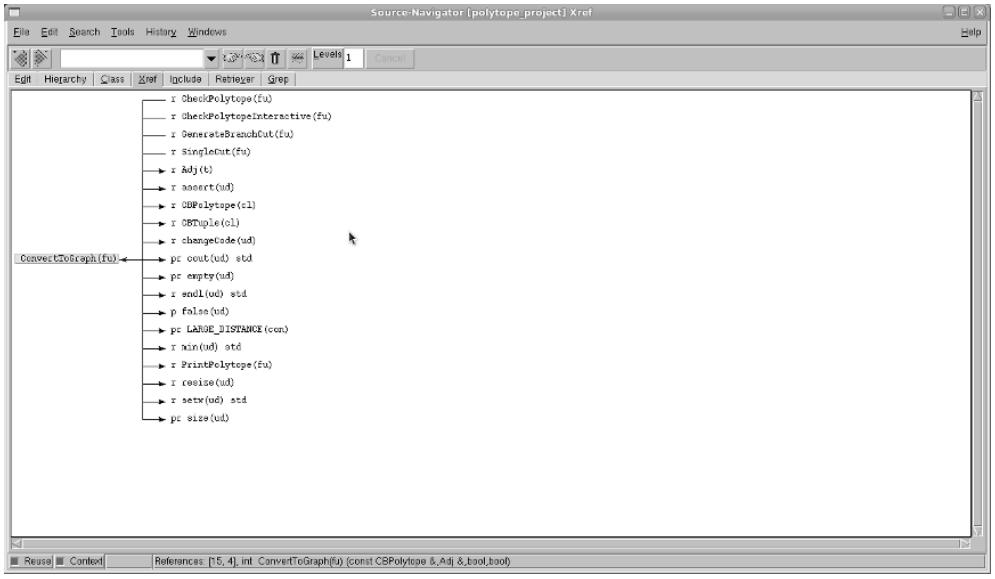



(a) SourceNavigator file view

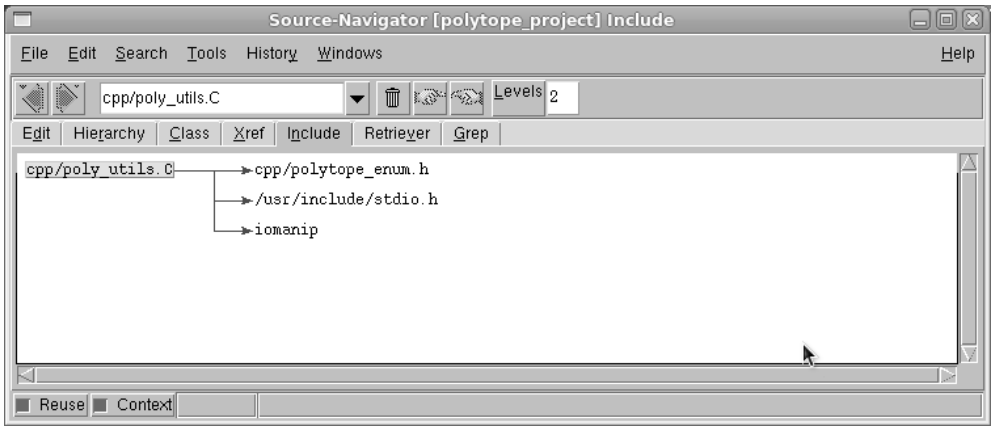



(b) SourceNavigator data view

**Fig. 3.17** SourceNavigator : project management for code




(a) SourceNavigator call graph




(b) SourceNavigator file inclusion graph

**Fig. 3.18** SourceNavigator : project management for code

parses the code, infers the dependencies and creates the project database. As shown in Figure 3.18, source code can be analyzed for dependencies within SourceNavigator. Even though SourceNavigator has a code editor, it is also possible to invoke an external editor such as Emacs (see Section 3.6.1) to edit the source code. As and when the source code is modified, SourceNavigator updates the project database to keep the dependencies current.

## 3.12 Profilers

In this section we discuss code profiling. It is widely believed that "premature optimization is evil", and even when the code is mature and stable, programmers have a tendency to make mistakes when figuring out which portion of the program is the performance bottleneck. Towards this end, automatic code profiling tools have been developed. The work flow for profiling, is to compile the source program with instrumentation in order to calculate execution counts, as well as measuring the compute time spent in individual functions. Then, the program should be executed on a *representative workload*. During this execution, the program may execute several times slower than normal (due to the measurement taking place). Thus, this technique of instrumented code profiling may not be suitable for all programs, such as those requiring real-time interaction, as the measurement itself would cause the program to execute a different code pattern than the normal workload.

But for many other programs, where the program is compute bound, automatic profiling can quickly identify code sections which should be optimized; either using better algorithms and data-structures, or by compiler settings such as loop unrolling, inlining and feedback directed optimization. The two common open-source profilers are (i) GNU profiler `gprof`, and (ii) Valgrind. The former is a code instrumentation based profiler which calculates the amount of time the program counter is located within the stack frame of a function and calculates cumulative program time using sampling. Valgrind, on the other hand is not only a code profiler, but also contains tools for memory error checking, cache trace analysis, and thread error checking. Valgrind can also analyze programs without code instrumentation. Valgrind actually simulates an instruction set and can thus perform much greater levels of introspection on the program. These two profilers are discussed in the following sections.

### *3.12.1 GNU profiler : `gprof`*

The process of code profiling using GNU `gprof` can be divided into two parts: (i) source code instrumentation and execution, and (ii) call graph display using the `gprof` program. To instrument the source code for profiling the `-pg` command-line argument to GNU `gcc` has to be used both during compilation as well as linking. While optimization settings can be left on, it is recommended to include debug symbols (using the `-ggdb` command-line argument). During the initial phase of the profiling, optimizations can be switched off also, as algorithmic optimizations are not usually affected by compiler optimizations, and in the first phase, we should strive to detect algorithmic outliers in the implementation. As stated above, the instrumented program should now be run on representative workloads. The runtime of this instrumented binary is higher than the non-instrumented code. It should also be noted that multi-threaded sampling is still (as of version 4.5.0 in the GNU compilers) not reliable, and multi-threaded execution should be switched off during profiling. But, we expect this problem to be addressed in the short term. On completion of the pro-

gram, the program writes out a file (usually called `gmon.out`) which contains the summary of the execution data. This file is subsequently analyzed using the `gprof` command.

#### 3.12.1.1 `gprof` - display call graph profile data

As we stated above, after execution of the instrumented binary we get a file, 'gmon.out' which is analyzed using `gprof`. The GNU profiler has a number of command-line options as listed below. The general command-line syntax for `gprof` is:

```
$gprof <options> <executable> <gmon.out>
```

`-A=symspec`: print annotated source listing, example:

```
       template <>
1468 -> inline int ChangeCode< CBTuple >( const CBTuple& v ) {
            return v.changeCode;
       }
...
Top 10 Lines:

     Line      Count

      145       1468
       89        404
       91         43
      151          3
      173          3
      447          3
      320          2
      454          2
       95          1
      103          1

Execution Summary:

       22   Executable lines in this file
       22   Lines executed
   100.00   Percent of the file executed

     1934   Total number of line executions
    87.91   Average executions per line
```

`-b`: be less verbose (the `brief` mode), see example shown below.
`-C`: print tally of functions and the number of time each function was called; for example,

```
stl_tree.h:249: \
(_ZNKSt23_Rb_tree_iteratorIiEdeEv:0x8051a46)
                            57004 executions
```

sorting the produced report file on the 3rd column gives us the function which was invoked the maximum number of times:

```
$sort -r -n -k 3 profile.txt | more
```

`-i`: display information about the profile data:

```
File 'gmon.out' (version 1) contains:
1 histogram record
1984 call-graph records
0 basic-block count records
```

`-p`: print the *flat profile* (see discussion on profile types below),
`-P`: no flat profile,
`--graph`: print the call graph analysis; for example:

```
                     Call graph (explanation follows)


granularity: each sample covers 4 byte(s)
                                    for 33.33% of 0.03 seconds

index % time    self  children    called     name
                                             <spontaneous>
[1]    100.0    0.00    0.03                   main [1]
                0.00    0.03 1/1             DP::Run() [2]
...
0.00    0.01    2290/4566 set<int, less<int> ::set() [8]
0.00    0.00    2290/4825 set<int, less<int> ::~set() [12]
```

`-r`: print suggested function ordering for this program; for example:

```
_ZNSt15insert_iteratorISt3setIiSt4lessIiESaIiEEEppEv
_ZSt16set_intersectionISt23_Rb_tree_const_iteratorIiES1
_ZNKSt23_Rb_tree_const_iteratorIiEneERKS0_
```

These C++ names can be demangled using the `c++filt` program as discussed in Section 13.5.1.3. Using this program we can convert the above names to C++ identifiers:

```
$c++file < file.txt |more
std::insert_iterator<set<int, less<int> >::operator++()
std::insert_iterator<set<int, less<int>> > > std:: \
          __copy_move<false, false, std::bidirectional_iterat
...
```

`-R mapfile`: print a suggested '.o' (object file) link line ordering based on profile data. The mapfile passed to the command-line can be generated using the `nm` (see Section 13.5.1.5) GNU binutils tool. The map file has the following syntax:

```
CP:08049690 T _init
CP:08049cc0 T _start
```

```
CP:08049cf0 T __gmon_start__
CP:0804a1cb T _ZlsRSoRK22PolyRep
CP:0804a97c T _ZN22PolyRep13AddToGraphicsEPS
CP:0804b0e4 T _ZN22PolyRep11ComputeRankEv
CP:0804b12a T _ZN22PolyRep19CalculatePropertiesEv
CP:0804dccc T _ZN13DPsor14parse_verticesEj
CP:0804eb1c T _ZN13DPsor22parse_vertices_reducedEj
CP:0804f406 T _ZN13DPsor12new_polytopeEv
CP:0804f424 T _ZN13DPsor3RunEv
CP:0804fe64 T main
```

The suggested ordering optimizes paging, TLB (translation lookahead buffer) and instruction cache behavior for the program. To use the `-R` option, the `-a` option is recommended.

`gprof` calculates the amount of time spent in each function and propagates this on the edges of the call graph to arrive at cumulative program times. Using `gprof` we can generate the following styles of reports:

1. Flat profile: displays how much time was spent in each function, and how many times that function was called. For each function it prints a tuple of information:
   a. percentage of total: running time used by the function,
   b. cumulative seconds: running sum of number of seconds,
   c. self seconds: number of seconds accounted by this function alone,
   d. number of calls: number of times this function was invoked,
   e. average number of milliseconds: spent in this function and descendants,
   f. name of the function: example is shown below.

```
Each sample counts as 0.01 seconds.
%   cumulative   self          self     total
time sec  sec   calls  Ts/call Ts/call  name
 0.0 0.0  0.0 22159596  0.0     0.0  ConvertToPolytope
```

2. Call graph profile: for each function, displays which other functions called the named function, which functions did the named function call, and also an estimate of how much time was spent in each sub-routine call.
3. Annotated source listing: is a copy of the source code, labeled with the execution count of each line (similar to GNU `gcov`, see Section 3.8).

To get a concise overview of the performance of the program, use the *flat profile*. To eliminate spurious function calls, use *call graph profile*, and for a systematic listing of execution counts alongwith the code, use the *annotated source listing*.

```
Each sample counts as 0.01 seconds.
  %   cumulativef             self     total
 time   seconds   calls   s/call   s/call  name
  8.96  0.12       1632     0.00     0.00  CBPolytope::init(bool)
  7.84  0.23     8426811     0.00     0.00  std::vector<std::vector
  6.72  0.32                              refine1
  5.97  0.40     8674249     0.00     0.00  std::vector<int>
  3.73  0.45     1960465     0.00     0.00  int const& std::min<int>
```

GNU profiler `gprof` is a versatile tool and can result in significant performance improvement. Consider the following sample of a *brief* profile output (part of the output have been edited to make the data fit in a reduced number of pages):

```
Each sample counts as 0.01 seconds.
 %   cumulative   self         self     total
time   seconds   seconds  calls s/call s/call  name
17.0 21.43 21.43      29879  0.0 0.0  CBPolytope::init(bool)
11.0 35.35 13.93 1016658200  0.0 0.0  vector<A>::operator[](uint)
 9.0 46.75 11.39 1034828918  0.0 0.0  vector<B>::operator[](uint)
 4.3 63.24 5.46   244592208  0.0 0.0  int const& std::min<int>
```

The `CBPolytope::init()` turned out to be an algorithmic issue, where the data-structure representing the polytope could simply be copied from a basic initialized polytope (and need not be initialized per polytope). The remaining are classic C++ member function and STL optimizations. In loops dealing with STL vectors it is common to write:

```
for(size_t i=0; i < vec.size(); ++i ) {
  // process vec[i]
}
```

Due to C++ semantics, the `vector<T>::size()` function is called per trip of the **`for`** loop. It is indeed much better to write the code as:

```
for( size_t i=0, e=vec.size(); i < e; ++i ) {
  // process vec[i]
}
```

In this version of the loop, `vector<T>::size()` is evaluated once. The programmer must make sure that this is the intended semantics of the loop. Similarly, writing the `min` function for references as an inline function will reduce function call overhead in that example.

## *3.12.2 Valgrind*

Valgrind is a instrumentation framework for building dynamic analysis tools. Valgrind includes: (i) cache and branch-prediction profiler, (ii) memory error detector, (iii) two thread error detectors, and (iv) a heap profiler.

### 3.12.2.1 Cachegrind, a cache and branch-prediction profiler

is the performance analyzer tool of Valgrind. It is invoked with the

```
Cachegrind, a cache and branch-prediction profiler
Copyright (C) 2002-2009, and GNU GPL'd, by Nicholas Nethercote et al.
Using Valgrind-3.5.0 and LibVEX; rerun with -h for copyright info
Command: ./poly_enum_i686 3 1000 9


I   refs:      306,463,674
I1  misses:        346,980
L2i misses:          2,563
I1  miss rate:        0.11%
L2i miss rate:        0.00%
```

```
D   refs:      165,173,691  (106,648,498 rd   + 58,525,193 wr)
D1  misses:        503,574  (    400,345 rd   +    103,229 wr)
L2d misses:         15,407  (      4,319 rd   +     11,088 wr)
D1  miss rate:         0.3% (        0.3%     +        0.1%  )
L2d miss rate:         0.0% (        0.0%     +        0.0%  )

L2 refs:           850,554  (    747,325 rd   +    103,229 wr)
L2 misses:          17,970  (      6,882 rd   +     11,088 wr)
L2 miss rate:          0.0% (        0.0%     +        0.0%  )
```

The `cachegrind` tool has the following options:

`--I1=<size>,<associativity>,<line size>`: specify size, associativity and line size of level 1 instruction cache,
`--D1=<size>,<associativity>,<line size>`: specify size, associativity, and line size of level 1 data cache,
`--L2=<size>,<associativity>,<line size>`: specify size, associativity, and line size of level 2 cache,
`--branch-sim=no|yes`: enables or disables collection of cache access trace.

#### 3.12.2.2 Callgrind, a call-graph generating cache profiler

A sample of the output from `callgrind` is shown below:

```
==23126==
==23126== Events    : Ir
==23126== Collected : 604392288
==23126==
==23126== I   refs:      604,392,288
```

#### 3.12.2.3 Memcheck, memory error detector

This is the default tool of Valgrind, and performs memory leak detection as well as memory errors. A sample output is shown below:

```
HEAP SUMMARY:
in use at exit: 11,852,753 bytes in 36,614 blocks
 total heap usage: 388,258 allocs, 351,644 frees,
                      22,409,570 bytes allocated

LEAK SUMMARY:
   definitely lost: 13,834 bytes in 172 blocks
   indirectly lost: 280,784 bytes in 3,455 blocks
   possibly lost: 7,431 bytes in 195 blocks
still reachable: 11,550,704 bytes in 32,792 blocks
        suppressed: 0 bytes in 0 blocks
Rerun with --leak-check=full to see details of
leaked memory

For counts of detected and suppressed errors,
rerun with: -v
ERROR SUMMARY: 0 errors from 0 contexts
(suppressed: 15 from 8)
```

The `memcheck` tool has the following options:

`--leack-check=<no|summary|yes|full>`: search for memory leaks when the program finishes. This argument specifies the details to be computed for the leak,
`--leak-resolution=<low|med|high>`: whether to use multiple back-traces to the same memory,
`--show-reachable=<yes|no>`: when disabled the checker will only state "definitely lost",
`--malloc-fill=<hexnumber>`: can aid in detecting uninitialized memory usage by filling allocated memory with prescribed bit-pattern,
`--free-fill=<hexnumber>`: check against use-after-free type errors.

#### 3.12.2.4 Helgrind, a thread error detector

Consider the thread-pool example of APR code (see Section 5.4) shown in Listing 5.3. We can analyze this code for data races, critical section errors, and other multi-threading problems by running the `helgrind` thread checker tool from Valgrind. We invoke the binary as:

```
valgrind -tool=helgrind ./a.out 2>thread_errors.txt
```

The output from the checker is shown below:

```
Helgrind, a thread error detector
Copyright (C) 2007-2009, and GNU GPL'd, by OpenWorks LLP et al.
Using Valgrind-3.5.0 and LibVEX; rerun with -h for copyright info
Command: ./thpool

Thread #1 is the program's root thread

Thread #2 was created
   at 0xA12DB8: clone (in /lib/libc-2.11.1.so)
   by 0xAE7FF2: pthread_create@@GLIBC_2.1
                          (in /lib/libpthread-2.11.1.so)
   by 0x400A192: pthread_create_WRK (hg_intercepts.c:229)
   by 0x400A225: pthread_create@* (hg_intercepts.c:256)
   by 0x4048871: apr_thread_create (thread.c:179)
   by 0x403125D: apr_thread_pool_create (apr_thread_pool.c:380)
   by 0x8048CB0: main (thpool.cpp:129)

Possible data race during read of size 4 at 0x41fb130 by thread #1
   at 0x403C2CE: apr_palloc (apr_pools.c:649)
   by 0x40487AC: apr_thread_create (thread.c:154)
   by 0x403125D: apr_thread_pool_create (apr_thread_pool.c:380)
   by 0x8048CB0: main (thpool.cpp:129)
 This conflicts with a previous write of size 4 by thread #2
   at 0x403C37B: apr_palloc (apr_pools.c:651)
   by 0x4030E2C: thread_pool_func (apr_thread_pool.c:222)
   by 0x40484B5: dummy_worker (thread.c:142)
   by 0x400A2A4: mythread_wrapper (hg_intercepts.c:201)
```

```
by 0xAE7AB4: start_thread (in /lib/libpthread-2.11.1.so)
by 0xA12DCD: clone (in /lib/libc-2.11.1.so)
```

A detailed analysis of this log can be helpful in uncovering race conditions, and other errors in multi-threaded applications. Valgrind can also check for *self modifying code*, which can be present in malicious programs. To enable this check we use the `--smc-check=<stack|all>` command-line option. Valgrind can be used to optimize the program's performance and check for memory leaks. There are a number of GUI frontends for Valgrind, one of which is `alleyoop`, as shown in Figure 3.19.

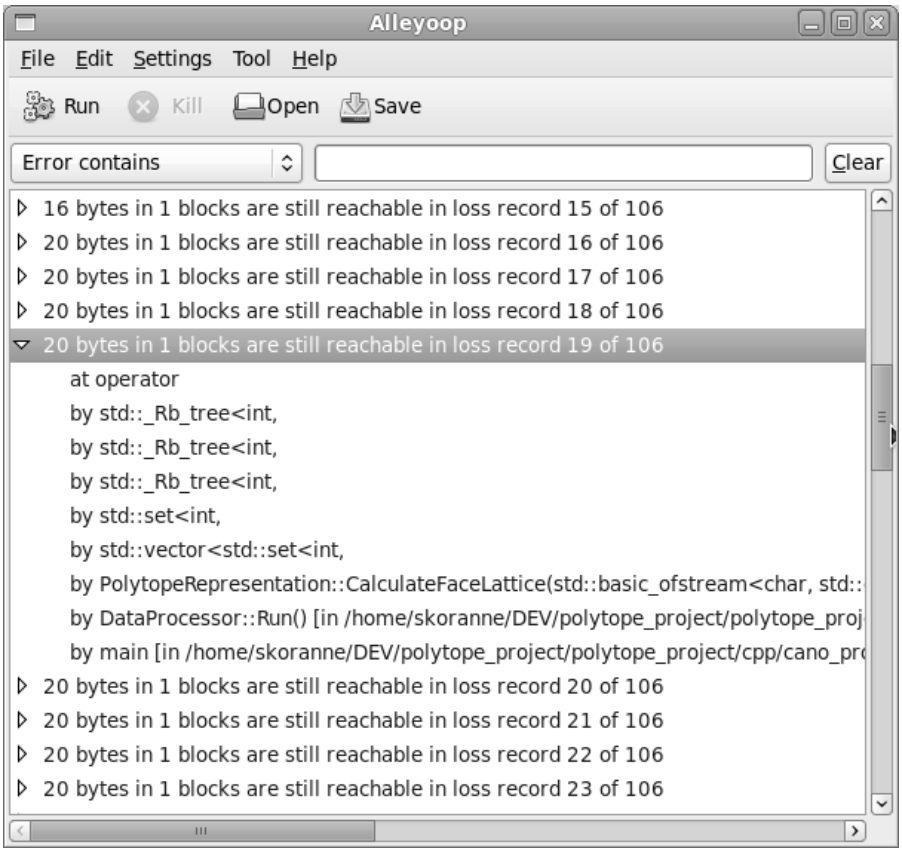



**Fig. 3.19** Alleyoop: GUI frontend for Valgrind

## 3.13 Conclusions

In this chapter we have discussed software construction tools. We discussed the venerable GNU Compiler Collection (GCC) compilers, source code configuration systems (CVS, SVN and git), as well as the GNU Build Tools. Automatic build man-

agement tools GNU `gmake` and SCons are described with examples. The Bugzilla defect tracking system is described; incidentally, this book had a Bugzilla page during its development for issue tracking. Source code editors and IDEs (including Emacs, Kdevelop, and Eclipse) are shown as well as debugging, documentation, and profiling tools.

In many ways, this chapter contains foundational material for starting an open-source project, and for contributing to an existing project. Reading the documentation for each of the tools mentioned in this chapter, reading their source code, and experimenting with it will provide valuable experience to the reader in developing and using open-source software in the subsequent chapters.